

TECLE

TAPOS

Tecle AI Platform Operating System

Knowledge Base — v1.0

16/07/2026

Sumário

Base Institucional

- Empresa
- Visão
- Competências

TAPOS — Tecle AI Platform Operating System

- Visão Geral
- Arquitetura
- Princípios
- Roadmap
- Tecnologias

Speech-AI

- Visão Geral
- Arquitetura
- Pipeline
- Componentes
- Provedores
- IA Local
- Motor de Execução
- Modelos de Dados
- Saídas
- Configuração
- Roadmap

Produtos

- Edital-AI
- Educa-AI
- Code-AI

Platform

- Runtime
- Storage
- Segurança
- Observabilidade
- API
- Gateway

Desenvolvimento

- Workspace
- Constituição

Padrões de Código

Claude Code

Fluxo de Desenvolvimento

Base Institucional

Empresa

O que é a Tecle

A Tecle é uma empresa de engenharia de software especializada em transformar Inteligência Artificial em produtos SaaS vendáveis, cobráveis e escaláveis. Não constrói modelos de IA do zero: constrói a ponte de engenharia — autenticação, cobrança, execução em escala, infraestrutura — que falta entre um protótipo de IA que funciona e um negócio de software que fatura.

A tese central da empresa é simples de enunciar e difícil de executar: **a maioria dos projetos de IA corporativa morre na transição entre o notebook que funciona e o SaaS que fatura**. Cada novo produto de IA tende a reconstruir, do zero, a mesma infraestrutura de negócio — autenticação de usuários, controle de quem pagou por quê, execução assíncrona para não travar sob carga, filas, workers, histórico, exportação de resultados. É trabalho de engenharia real, caro, e que não tem nada a ver com o diferencial de cada produto de IA em si.

A Tecle resolveu esse problema uma única vez, na camada de plataforma — a **TAPOS (Tecle AI Platform Operating System)** — e a partir dela já opera três produtos de IA completamente diferentes entre si sobre a mesma esteira de autenticação, assinatura, gateway de autorização e execução assíncrona.

O que a Tecle constrói

A plataforma

A **TAPOS** é o sistema nervoso central de todos os produtos: um backend SaaS em FastAPI e PostgreSQL que resolve, de forma genérica e reutilizável, autenticação e identidade, produtos e assinaturas, gateway central de autorização e execução assíncrona real via fila de mensagens e workers dedicados. Ver 01-tapos/overview.md.

Os produtos

Sobre essa plataforma, a Tecle opera hoje três verticais de IA aplicada, e reserva uma quarta para os próximos ciclos:

- **[Speech-AI](#)** — inteligência de fala: transforma roteiros de texto em narração de áudio, com análise linguística prévia (idioma, complexidade, ritmo, pausas, perfil de voz) antes da síntese. É o produto mais maduro e o primeiro integrado de ponta a ponta na plataforma.
- **[Edital-AI](#)** — automação de licitações públicas brasileiras: extrai de forma determinística prazos, itens, lotes e documentos de habilitação de editais, e usa IA local para gerar resumo executivo, riscos e um score de conformidade.

- [Code-AI](#) — conversor universal de documentos corporativos (PDF, Word, Excel, PowerPoint, imagens) para Markdown limpo, pronto para consumo por LLMs e pipelines de RAG, com economia estimada de 70–85% em tokens.
- **Educa-AI** (reservado) — quarta vertical planejada, voltada a educação financeira e acompanhamento de mercado. Ver [03-products/educa-ai.md](#) para o estágio atual.

Modelo de negócio

Cada produto é uma vertical de receita independente, vendida como assinatura sobre a mesma base de plataforma. O modelo de **produtos e assinaturas** já está implementado na TAPOS: cada usuário tem assinaturas ativas ou inativas por produto, o que habilita — sem nenhuma reengenharia — a venda de módulos separadamente. O ganho de escala não está em cada produto isoladamente, mas no fato de que o próximo produto lançado herda toda a infraestrutura de autenticação, cobrança e execução no primeiro dia.

Estágio atual

A Tecle está em estágio inicial (seed), com a plataforma tecnicamente validada e em produção local:

- infraestrutura operacional (Docker, PostgreSQL, Redis, RabbitMQ, MinIO, Qdrant, Ollama);
- autenticação, produtos, assinaturas, gateway e execução assíncrona implementados e testados (baseline v1.0.0);
- Speech-AI integrado de ponta a ponta; Edital-AI e Code-AI também conectados ao gateway central;
- Edital-AI já processou editais públicos reais de prefeituras e órgãos brasileiros, com resultados auditáveis.

As lacunas conhecidas — multi-tenancy completo, paralelização de processamento, hardening de segurança no upload de arquivos, pipeline de deploy contínuo e definição final do modelo de cobrança — estão mapeadas e são extensões naturais da arquitetura existente, não redesenhos. Ver [01-tapos/roadmap.md](#).

Ver também

- [vision.md](#) — a tese de plataforma e a filosofia de engenharia por trás da Tecle
- [competencies.md](#) — as competências técnicas e de domínio que sustentam essa tese

Visão

A tese de plataforma

TAPOS não é um produto de Inteligência Artificial. **É o sistema operacional sobre o qual produtos de Inteligência Artificial se transformam em negócio.**

Essa frase resume a visão da Tecle. O mercado está cheio de protótipos de IA que funcionam tecnicamente e morrem comercialmente, porque conseguir fazer a IA responder deixou de ser o problema difícil — o problema difícil passou a ser transformar isso em algo vendável, cobrável e escalável. Isso exige autenticação, controle de quem pagou por quê, execução assíncrona sob carga, filas, workers, histórico e exportação de resultado: infraestrutura de negócio que nada tem a ver com o diferencial de cada produto de IA, mas que sem ela nenhum produto vira empresa.

A Tecle escolheu resolver esse problema **uma única vez, na camada de plataforma**, e reaproveitá-lo em cada nova vertical. O resultado observável disso é que voz (Speech-AI), documentos jurídicos de licitação (Edital-AI) e conversão de arquivos corporativos (Code-AI) — três produtos sem nenhuma semelhança funcional entre si do ponto de vista do usuário final — rodam hoje sobre exatamente a mesma esteira de autenticação, assinatura, gateway de autorização e execução assíncrona.

O que isso muda

Isso muda fundamentalmente a economia de lançar um quarto, um quinto, um sexto produto. A próxima vertical de IA que a Tecle lançar não precisa reconstruir autenticação, cobrança ou infraestrutura de execução — ela herda tudo isso no primeiro dia, e o time de engenharia investe cem por cento do seu tempo no que realmente diferencia o novo produto. Essa é, literalmente, a definição de uma plataforma. É também o motivo pelo qual já existe uma quarta vertical, o Educa-AI, reservada na estrutura da TAPÓS para os próximos ciclos.

Princípio: Inteligência Artificial local sempre que possível

Há um segundo fio condutor entre os produtos da Tecle, tão importante quanto o primeiro: a filosofia de **usar IA local sempre que a tarefa permitir**, minimizando dependência de provedores de IA em nuvem cobrados por token.

- O Speech-AI foi desenhado com uma camada de abstração de provedores de síntese de voz, para não ficar refém de um único fornecedor.
- O Edital-AI usa extração determinística (leitura de tabelas nativas, reconhecimento de padrões) para todos os dados críticos, e reserva IA — rodando localmente via Ollama — apenas para a camada de análise (resumo, riscos, score), com um mecanismo de segurança determinístico caso a IA falhe.

Essa não é apenas uma escolha técnica — é uma escolha de margem. Enquanto muitos concorrentes de mercado operam como interfaces finas sobre APIs de terceiros, pagando por token a cada execução, a arquitetura da Tecle protege a margem bruta do negócio da variação de custo de provedores externos.

Princípio: confiabilidade antes de sofisticação

Onde a IA pode falhar, o pipeline não pode travar. O Edital-AI é o exemplo mais claro dessa regra: se a camada de IA falhar ou estiver indisponível, um mecanismo de segurança determinístico garante que o processo de análise continue até o fim. A Tecle prioriza sistematicamente confiabilidade sobre sofisticação sempre que as duas estiverem em tensão.

Honestidade sobre o estágio

A visão da Tecle inclui ser transparente sobre o que já está pronto e o que ainda não está. Nenhum dos produtos é vendido como mais maduro do que é:

- o Speech-AI é o mais validado, já rodando em produção síncrona e assíncrona;
- o Edital-AI já processa editais reais, mas opera hoje como piloto validado de single-tenant (um edital por vez em toda a instalação);
- o Educa-AI é explicitamente reservado para o futuro, sem implementação ainda.

As lacunas entre piloto validado e produto comercial em escala — multi-tenancy, paralelização, hardening de segurança, deploy contínuo, modelo de cobrança final — são conhecidas, mapeadas e tecnicamente tratáveis. Nenhuma delas exige redesenhar a plataforma: todas são extensões naturais de uma arquitetura já construída, desde o primeiro dia, pensando em desacoplamento e escala.

Onde a Tecle quer chegar

O objetivo de longo prazo é operar uma plataforma multi-produto de IA que atenda centenas de clientes simultaneamente, com cada nova vertical de IA lançada como um incremento de receita sobre uma fundação já paga — nunca como um novo projeto de engenharia começado do zero.

"A Tecle não está pedindo a um investidor para apostar em uma ideia. Está mostrando uma plataforma que já processa editais públicos reais, já converte documentos corporativos reais, e que gerou o próprio áudio do seu discurso institucional através do seu primeiro produto em produção." — [Documento para Investidores-Anjo, TAPOS v1.0](#)

Ver também

- [company.md](#) — o que a Tecle é e constrói hoje

- [01-tapos/principles.md](#) — como essa visão se traduz em princípios de engenharia concretos

Competências

A Tecle combina três camadas de competência que, juntas, sustentam a tese de plataforma: engenharia de infraestrutura SaaS, aplicação prática de Inteligência Artificial e conhecimento de domínio vertical. Nenhuma das três sozinha diferenciaria a empresa — é a combinação das três, replicada de forma consistente entre produtos, que faz da TAPOS uma plataforma e não uma coleção de projetos.

1. Engenharia de plataforma SaaS

- **Autenticação e identidade:** cadastro, login e sessão protegida por JWT, com senhas protegidas por hash bcrypt.
- **Modelo de produtos e assinaturas:** cada vertical de IA como um produto com identidade própria (slug), com assinaturas ativas/inativas por usuário — infraestrutura de cobrança modular pronta antes mesmo da decisão comercial de precificação.
- **Gateway central de autorização:** nenhum produto é exposto diretamente ao usuário final; todo acesso passa por uma camada única que valida token e assinatura antes de liberar qualquer execução.
- **Execução assíncrona real:** fila de mensagens (RabbitMQ) com worker dedicado por produto, status consultável do estado "na fila" até "concluído" ou "falhou" — capacidade de escalar de um usuário de teste a centenas de execuções simultâneas sem travar a experiência de ninguém.
- **Isolamento por produto:** cada produto roda em ambiente Python isolado (subprocesso independente), comunicando-se com a plataforma central por um contrato simples de entrada/saída em JSON — um produto pode evoluir, quebrar ou ser reescrito sem colocar em risco os demais.

Ver [01-tapos/architecture.md](#) e [04-platform/gateway.md](#) para o detalhamento técnico.

2. Aplicação prática de Inteligência Artificial

A Tecle não constrói modelos de fundação — aplica IA (local, em nuvem ou determinística, conforme o problema exigir) dentro de um pipeline de produto confiável:

- **Síntese e inteligência de fala** (Speech-AI): análise linguística prévia — detecção de idioma, complexidade, ritmo de leitura, pausas e seleção de perfil de voz — antes da conversão texto→áudio, com arquitetura de provedores plugável.
- **IA local para análise de documentos** (Edital-AI): modelo rodando via Ollama para resumo executivo, identificação de risco/oportunidade e score de conformidade, sempre com fallback determinístico caso a IA falhe — a extração de dados críticos em si é 100% determinística (leitura de tabelas nativas, regex e heurísticas), não dependente de IA.
- **OCR e conversão de documentos para consumo por LLM** (Code-AI): normalização de PDF, Word, Excel, PowerPoint e imagens digitalizadas em Markdown limpo, reduzindo significativamente o consumo de tokens em pipelines de IA e RAG corporativos.

A regra que atravessa os três produtos: **usar IA onde ela gera diferencial real, e determinismo onde a confiabilidade importa mais do que a sofisticação.**

3. Conhecimento de domínio vertical

- **Compras públicas brasileiras:** entendimento profundo da estrutura de um edital de licitação — modalidades, tabelas de itens/lotos, documentos de habilitação, prazos críticos (abertura, impugnação, recurso) — validado contra editais reais de prefeituras e órgãos públicos.
- **Produção de conteúdo em áudio corporativo:** treinamentos, e-learning, apresentações institucionais, acessibilidade e comunicação multilíngue.
- **Engenharia de dados para IA corporativa:** entendimento de como equipes de dados constroem pipelines de RAG e por que o formato de entrada (não apenas o modelo) determina custo e qualidade.

4. Disciplina de engenharia e operação com IA como parceira de desenvolvimento

A própria forma como a Tecle constrói é uma competência: desenvolvimento incremental e disciplinado, um módulo por vez, testado e commitado antes do próximo passo, usando o **Claude Code** como assistente de engenharia orientado por contexto persistente (`.claude/workspace.md` , `.claude/constitution.md`). Essa disciplina é o que permitiu levar a plataforma de autenticação básica até gateway multi-produto com execução assíncrona em ciclos curtos e validados. Ver [05-development/development-flow.md](#).

Ver também

- [company.md](#)
- [vision.md](#)
- [01-tapos/technologies.md](#) — stack técnico concreto por trás dessas competências

TAPOS — Tecle AI Platform Operating System

Visão Geral

O que é

A **TAPOS (Tecle AI Platform Operating System)** é a plataforma de engenharia da Tecle para construção e operação de produtos SaaS baseados em Inteligência Artificial. Ela organiza desenvolvimento, runtime e infraestrutura de forma modular, desacoplada e escalável, permitindo evolução incremental e controle técnico completo.

Não é, em si, um produto de IA — é a fundação de negócio (autenticação, produtos, assinaturas, gateway de autorização, execução assíncrona) sobre a qual produtos de IA se transformam em receita recorrente, sem que cada novo produto precise reconstruir essa infraestrutura do zero. Ver a tese completa em [../00-tecle/vision.md](https://00-tecle/vision.md).

Objetivos da plataforma

- suportar múltiplos produtos SaaS independentes;
- centralizar autenticação e controle de acesso;
- padronizar desenvolvimento e operação;
- reduzir custo e dependência de fornecedores externos de IA;
- utilizar IA local (Ollama) e arquitetura híbrida sempre que possível;
- garantir reprodutibilidade do ambiente.

Estado atual (baseline v1.0.0)

Segundo `changelog.md` e confirmado por leitura direta do código do backend (`platform/saas-backend/`):

- autenticação JWT (`bcrypt` + `python-jose`);
- modelo de Produtos e Assinaturas;
- Autorização central (verificação de assinatura ativa antes de qualquer execução);
- Product Gateway (padrão de autorização + despacho aplicado a todas as rotas de produto);
- execução assíncrona real via RabbitMQ + worker dedicado por produto;
- Speech-AI integrado de ponta a ponta (síncrono e assíncrono); Edital-AI e Code-AI também conectados ao mesmo gateway.

Este é o estado real e validado — não uma proposta de arquitetura no papel. Um usuário autenticado hoje consegue assinar um produto, enviar um arquivo, disparar a execução (síncrona ou assíncrona) e consultar o resultado, para os três produtos implementados.

Nota de consistência: `.claude/current-phase.md` descreve um estágio muito anterior ("SaaS Backend Initialization", construindo o endpoint `/health`) e está desatualizado — não deve ser tratado como fonte de verdade sobre o estágio atual. `changelog.md` e o histórico em `tasks/saas/` são as fontes corretas. Ver [../05-development/claude-code.md](https://github.com/ta-pos/ta-pos/blob/main/05-development/claude-code.md).

Camadas

```
Cliente
↓
SaaS Backend (FastAPI) — autenticação, produtos, assinaturas, gateway
↓
Produtos (speech-ai, editai-ai, code-ai — cada um isolado em seu próprio .venv)
↓
Infraestrutura (/data/platform — PostgreSQL, Redis, RabbitMQ, MinIO, Qdrant,
Ollama)
```

Duas camadas de execução hoje

1. **Laboratório local** (VM Ubuntu) — ambiente de desenvolvimento e execução, VSCode + Claude Code, backend SaaS em FastAPI, infraestrutura Docker persistente.
2. **Evolução futura para VPS** — publicação e operação da camada SaaS em produção, espelhando o ambiente local de forma controlada. Ainda não implementada.

Ver também

- [architecture.md](#) — detalhamento técnico de cada camada
- [principles.md](#) — princípios de engenharia que regem a plataforma
- [roadmap.md](#) — histórico de construção e próximos passos
- [technologies.md](#) — stack técnico completo

Arquitetura

Backend SaaS

Um único serviço FastAPI (`platform/saas-backend/app/main.py`), estruturado em routers por recurso:

```
app/
├── main.py          → cria a app, monta routers, expõe /health e /ui
├── db.py            → engine/sessão SQLAlchemy
├── models.py        → User, Product, Subscription, Job, EditalAnalise
├── security.py      → hash bcrypt + JWT
├── deps.py          → get_current_user, get_product_access
├── routes/
│   ├── auth.py      → /auth/register, /auth/login
│   ├── users.py     → /users/me
│   ├── products.py  → /products, gateway de produtos
│   └── subscriptions.py → /subscriptions
├── jobs/
│   ├── publisher.py → publish_job() via pika/RabbitMQ
│   ├── routes.py    → GET /jobs/{job_id}
│   └── schemas.py
└── products/
    ├── speech_ai_adapter.py
    ├── code_ai_adapter.py
    └── edital_ai_adapter.py
```

Ver [../04-platform/api.md](#) para a lista completa de endpoints e [../04-platform/gateway.md](#) para o mecanismo de autorização.

Modelo de dados central

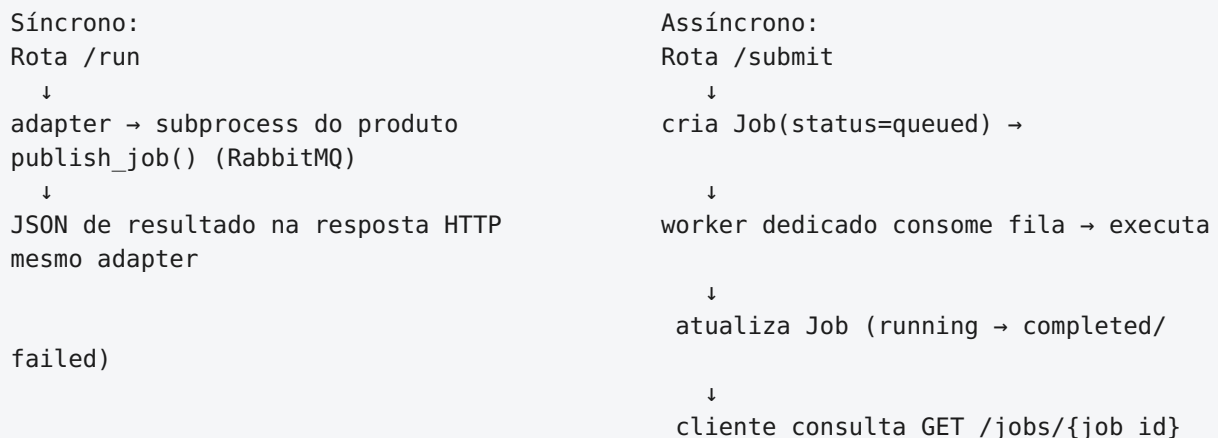
```
User —< Subscription >— Product
User —< Job                      (execução assíncrona, qualquer produto)
User —< EditalAnalise           (histórico específico do edital-ai)
```

`Product.slug` é o identificador estável de cada vertical (`speech-ai` , `edital-ai` , `code-ai`);
`Subscription.is_active` habilita, sem nenhuma reengenharia, um modelo comercial de módulos vendidos separadamente.

Isolamento de produto por subprocesso

Cada produto roda em seu próprio ambiente Python (`.venv` isolado), como subprocesso independente, comunicando-se com a plataforma por um contrato simples de entrada/saída em JSON (padrão `facade/runner/schemas/cli`, ver [../05-development/coding-standards.md](https://github.com/ta-pos/ta-pos/blob/main/docs/05-development/coding-standards.md)). Isso significa que um produto pode evoluir, quebrar ou ser reescrito em outra tecnologia sem colocar em risco os demais nem o núcleo da plataforma.

Execução: síncrona e assíncrona



Detalhado em [../04-platform/gateway.md](https://github.com/ta-pos/ta-pos/blob/main/docs/04-platform/gateway.md).

Infraestrutura de runtime

```
/data/platform
├─ infra/      → docker-compose e serviços
├─ runtime/    → execução de containers
└─ storage/    → persistência (db, redis, rabbitmq, minio, qdrant, models,
portainer)
```

Containers ativos hoje (verificado): `postgres:15`, `redis:7-alpine`, `rabbitmq:3-management`, `qdrant`, `minio`, `ollama`, `open-webui`, `portainer`. O backend e os workers rodam como processos diretos no host (não containerizados). Ver [../04-platform/runtime.md](https://github.com/ta-pos/ta-pos/blob/main/docs/04-platform/runtime.md) e [../04-platform/storage.md](https://github.com/ta-pos/ta-pos/blob/main/docs/04-platform/storage.md).

Fluxo de autenticação

```
Cliente → POST /auth/login → backend valida usuário/senha → JWT gerado
      → cliente usa Authorization: Bearer <token>
      → GET /users/me → backend valida token e retorna perfil
```


Ver também

- [overview.md](#) — visão geral e estado atual da plataforma
- [../04-platform](#) — detalhamento operacional de cada camada (runtime, storage, segurança, API, gateway)
- [technologies.md](#) — stack completo

Princípios

Estes princípios são a versão de plataforma da [Constituição de Engenharia](#) (`.claude/constitution.md`), que é a fonte normativa vigente — `governance/constitution/` existe apenas como diretório reservado, ainda vazio.

1. Modularidade

Backend SaaS, produtos e infraestrutura são componentes independentes, sem dependência direta entre si.

2. Desacoplamento

- Produtos **nunca** acessam o banco de dados diretamente — toda comunicação é via API ou via o contrato `facade/runner/schemas/cli`.
- Infraestrutura não é exposta ao domínio de negócio.

Evidência concreta: os três produtos rodam como subprocessos em `.venv` isolados, comunicando-se com o backend apenas por JSON via stdout (ver [architecture.md](#)).

3. Simplicidade

Evitar complexidade desnecessária; implementar apenas o necessário; não antecipar problemas sem evidência. Observado na prática: rotas FastAPI simples, sem camada de serviço ou repositório intermediária (ver [../05-development/coding-standards.md](#)).

4. Incrementalidade

Um módulo por vez, testado antes de avançar, commit constante — como princípio de planejamento. As 13 tarefas em `tasks/saas/` (005 a 015) mostram exatamente esse encadeamento incremental no nível de escopo. Na prática de versionamento em Git, porém, esse princípio **não foi seguido à risca**: todo o histórico até a v1.0.0 está em um único commit — ver [roadmap.md](#) e [../05-development/development-flow.md](#) para o registro honesto dessa divergência.

5. Testabilidade

Código deve ser executável isoladamente e validável sem depender de contexto implícito. Testes automatizados existem para o backend (`platform/saas-backend/tests/`); os produtos são validados principalmente por critérios de aceite manuais (`tasks/saas/*/acceptance.md`) — ainda não há suíte de testes automatizados por produto.

6. Reprodutibilidade

Ambiente reconstruível do zero: infraestrutura declarativa (Docker Compose), versionamento completo, sem dependências ocultas. `start_tapos_env.sh` automatiza esse bootstrap.

7. Persistência correta

Dados nunca dentro de containers — sempre em `/data/platform/storage`, fora do controle de versão. Verificado diretamente no sistema de arquivos (ver [../04-platform/storage.md](#)).

Regra de arquitetura (camadas)

```
User
↓
SaaS Backend
↓
Products
↓
Infrastructure
```

Práticas proibidas

- acesso direto ao banco por produtos;
- lógica de negócio dentro da infraestrutura;
- hardcode de configurações críticas (nota: o segredo padrão de JWT em desenvolvimento, `"dev-secret-key"`, é um risco a corrigir antes de produção — ver [../04-platform/security.md](#));
- implementar múltiplas features sem validação.

Ver também

- [../05-development/constitution.md](#) — o texto normativo completo
- [architecture.md](#) — onde estes princípios se manifestam em código
- [roadmap.md](#) — onde a prática já divergiu do princípio, e o que fazer a respeito

Roadmap

Histórico de construção (tasks/saas, 005 → 015)

Tarefa	Entregue
005-password-hash	Hash de senha com bcrypt
006-register-hash	Endpoint de registro de usuário
007-login	Login com validação de credenciais
008-jwt	Geração de JWT
009-protected-route	Rota protegida (/users/me) com JWT
010-user-profile	Perfil de usuário enriquecido
011-products	Modelo de produtos (slug por vertical)
012-subscriptions	Assinaturas ativas/inativas por produto
013-authorization	Autorização central (produto + assinatura)
014-product-gateway	Gateway único de acesso a produtos
014a-speech-ai-facade	Contrato facade/runner/schemas/cli do Speech-AI
014b-tapos-speech-ai-integration	Speech-AI integrado de ponta a ponta na plataforma
015-async-speech-ai-execution	Execução assíncrona real via fila e worker

Esse encadeamento é evidenciado nas próprias notas de tarefa: 007-login adia explicitamente "adicionar JWT" para a tarefa 008; 013-authorization adia "integração com speech-ai" e "integração com edital-ai" para 014a / 014b. O escopo estreito e sequencial foi seguido de forma consistente no planejamento.

Divergência conhecida: disciplina de commit

A constituição prescreve commit constante, um módulo por vez. Na prática, todo o histórico do repositório até esta documentação está em um único commit (a48b7b5 – TAP0S v1.0.0 - operational SaaS baseline), reunindo o trabalho de todas as 12 tarefas. O princípio não foi invalidado — precisa ser aplicado de forma mais rigorosa a partir daqui. Ver [../05-development/development-flow.md](https://github.com/ta-pos/ta-pos/blob/main/.github/development-flow.md).

Riscos conhecidos e lacunas mapeadas (da perspectiva de investimento)

Descritos com transparência no [documento para investidores-anjo](#) e confirmados tecnicamente pela pesquisa desta documentação:

- **isolamento completo de dados por cliente** (multi-tenancy) — hoje há isolamento por usuário, não por organização/cliente; o `edital-ai`, em particular, tem um único slot de "edital em foco" por instalação inteira;
- **paralelização do processamento assíncrono** — a fila e o worker já existem, mas a capacidade de processar múltiplos usuários verdadeiramente em paralelo, em escala, ainda não foi validada sob carga;
- **hardening de segurança no recebimento de arquivos** — sem limite de tamanho/timeout de upload, sem sanitização adicional (ver [../04-platform/security.md](#));
- **pipeline de implantação contínua em produção** — hoje o ambiente é local (VM Ubuntu); a evolução para VPS/produção ainda não foi construída;
- **definição final do modelo de cobrança** — a infraestrutura de assinatura (produtos + assinaturas ativas/inativas) já existe; falta apenas a decisão comercial entre cobrança por análise, por volume ou por assinatura fixa.

Nenhum desses itens exige redesenhar a plataforma — são extensões naturais de uma arquitetura já construída pensando em desacoplamento e escala desde o primeiro dia.

Próximos passos técnicos observados no código

- consolidar a checagem de assinatura duplicada entre `routes/products.py::_require_active_subscription` e `deps.py::get_product_access` (ver [../04-platform/gateway.md](#));
- conectar os workers de `edital-ai` e `code-ai` ao script de bootstrap automático (`start_tapos_env.sh`), hoje limitado ao worker do Speech-AI;
- migrar artefatos de produto de disco local para object storage (MinIO), hoje reservado mas não usado;
- popular `.claude/skills`, `agents`, `rules`, `memory` — reservados desde o início e liberados para uso "após estabilidade do backend SaaS", condição já atendida pela baseline v1.0.0 (ver [../05-development/claude-code.md](#));
- corrigir o valor padrão inseguro do segredo JWT (`dev-secret-key`) antes de qualquer exposição em produção.

Ver também

- [overview.md](#) — estado atual consolidado
- [../00-tecle/vision.md](#) — por que estas lacunas são vistas como oportunidade, não como bloqueio

- [../05-development/development-flow.md](#) — o ciclo de trabalho por trás deste histórico

Tecnologias

Backend SaaS (`platform/saas-backend/requirements.txt`)

Tecnologia	Papel
FastAPI	framework web do backend
Uvicorn	servidor ASGI
SQLAlchemy	ORM
psycopg2-binary	driver PostgreSQL
python-jose	geração/validação de JWT
passlib[bcrypt] + bcrypt<4.1	hash de senha
Pydantic	schemas de request/response
python-multipart	upload de arquivos (<code>UploadFile</code>)
pika	cliente RabbitMQ (publicação de jobs)
pytest (dev)	testes automatizados do backend

Infraestrutura local (`/data/platform` , containers Docker verificados ativos)

Serviço	Imagem	Papel
PostgreSQL	<code>postgres:15</code>	dados estruturados
Redis	<code>redis:7-alpine</code>	cache/sessões (reservado, ainda não usado ativamente)
RabbitMQ	<code>rabbitmq:3-management</code>	fila de mensagens para execução assíncrona
MinIO	<code>minio/minio</code>	armazenamento de objetos (reservado)
Qdrant	<code>qdrant/qdrant</code>	banco vetorial (reservado, uso futuro em RAG)
Ollama	<code>ollama/ollama</code>	runtime de IA local (usado hoje pelo <code>editai</code> , modelo <code>mistral</code>)
Open WebUI	<code>ghcr.io/open-webui/open-webui</code>	interface de administração/teste de modelos Ollama
Portainer	<code>portainer/portainer-ce</code>	administração dos containers Docker

Dependências por produto

- **speech-ai:** `edge-tts` , `langdetect` , `python-dotenv` (ver [../02-speech-ai](#));

- **editai-ai:** `pdfplumber`, `pytesseract`, `pdf2image`, `python-docx`, `openpyxl`, `reportlab`, `requests` (+ Tesseract OCR e Poppler como dependências de sistema; `jinja2` / `python-pptx` presentes mas não usadas — ver [../03-products/editai-ai.md](https://ta-pos.com/03-products/editai-ai.md));
- **code-ai:** `pdfplumber`, `python-docx`, `pandas`, `python-pptx`, OCR via `pytesseract` / `pdf2image` (ver [../03-products/code-ai.md](https://ta-pos.com/03-products/code-ai.md)).

Ambiente de desenvolvimento

- VSCode remoto (SSH) + Claude Code, operando a partir da raiz única `/workspace/tecle`;
- `.editorconfig` na raiz: indentação de 4 espaços, sem outras regras de estilo declaradas;
- sem linter/formatter configurado (nenhum `ruff`, `flake8`, `pylint`, `mypy` / `pyright` encontrado no repositório) — ver [../05-development/coding-standards.md](https://ta-pos.com/05-development/coding-standards.md);
- controle de versão: Git, com o desenvolvimento disciplinado por tarefa documentado em `tasks/saas/`.

Automação e infraestrutura como código

`automation/` reúne os diretórios reservados para Terraform, Ansible, Packer, scripts Bash/Python e GitHub Actions — nenhum deles populado ainda; a automação real hoje é o script `start_tapos_env.sh` na raiz do workspace.

Ver também

- [architecture.md](https://ta-pos.com/architecture.md) — como estas tecnologias se conectam
- [../04-platform/runtime.md](https://ta-pos.com/04-platform/runtime.md) — o runtime desses serviços em detalhe
- [../05-development/coding-standards.md](https://ta-pos.com/05-development/coding-standards.md) — convenções de código observadas

Speech-AI

Visão Geral

O que é

O Speech-AI é o primeiro produto integrado à TAPOS de ponta a ponta, e o mais maduro em validação técnica dentro da plataforma. Ele recebe um roteiro de texto e o transforma em áudio narrado — mas seu diferencial não é apenas converter texto em voz.

Antes de gerar qualquer áudio, o Speech-AI analisa o conteúdo: detecta automaticamente o idioma (português, inglês, espanhol ou francês), avalia a complexidade linguística, calcula o ritmo de leitura ideal, define pausas e escolhe o perfil de voz mais adequado ao conteúdo. Só então o texto é transformado em narração. Essa camada de análise prévia é o que a Tecle chama de **Speech Intelligence**, e é o que separa o produto de um simples conversor de texto em voz.

Problema que resolve

Produção de áudio narrado hoje exige ou gravação humana (cara, lenta, difícil de manter atualizada) ou ferramentas de texto-em-voz genéricas que ignoram estrutura, ritmo e adequação de tom ao conteúdo. O Speech-AI insere uma camada de inteligência entre o texto e a síntese, para que o resultado soe natural e adequado ao contexto — sem depender de locução humana para cada atualização de conteúdo.

Casos de uso

- Narração de treinamentos corporativos e conteúdos de e-learning
- Geração de áudio para apresentações institucionais
- Acessibilidade — leitura de documentos
- Comunicação corporativa em múltiplos idiomas
- Criação de conteúdo em escala (validado com um texto de livro inteiro, +29 mil linhas, processado ponta a ponta sem falhas)

O mercado global de tecnologias de conversão de texto em voz está estimado entre US\$ 4–6 bilhões em 2026, com crescimento anual de dois dígitos, impulsionado por assistentes de voz, plataformas de aprendizado digital e automação de atendimento.

Prova viva

O próprio [documento institucional para investidores](#) da Tecle foi narrado pelo Speech-AI processando seu próprio texto através do mesmo pipeline de produção descrito nesta documentação — a prova de conceito é o meio pelo qual a mensagem chega ao ouvinte.

Maturidade

Pipeline central (análise de texto → inteligência de fala → construção de narração → síntese via Edge TTS) é real, funcional e evidenciado por logs e artefatos de execução reais. A integração com a TAPOS (execução síncrona e assíncrona via fila e worker dedicado) também está implementada e operacional. Partes da visão de produto — um segundo pipeline de SSML mais avançado e uma camada de IA local para nuances de emoção/intenção — já têm código-base parcial ou planejamento detalhado, mas ainda não estão em produção. Ver [roadmap.md](#).

Ver também

- [architecture.md](#) — como os componentes se conectam
- [pipeline.md](#) — o fluxo de processamento passo a passo
- [../00-tecle/vision.md](#) — o papel do Speech-AI na tese de plataforma da Tecle

Arquitetura

Camadas

app/ ↓	→ orquestrador standalone (uso via `python main.py`)
pipeline/ Paragraph)	→ texto bruto → modelo de domínio (Presentation/Slide/Paragraph)
↓	
services/	→ Speech Intelligence + orquestração de síntese
↓	
providers/	→ abstração de motor de TTS (hoje: Edge TTS)
↓	
models/	→ dataclasses de domínio compartilhadas por todas as camadas
↓	
config/	→ settings.json + voices.json, carregados por ConfigManager

`config.config_manager.ConfigManager` é construído uma vez e passado adiante para `TextAnalyzer`, `NarrationBuilder` e `SpeechBuilder`; internamente ele possui um `VoiceManager` (`services/voice_manager.py`), carregado a partir de `config/voices.json`.

Pontos de entrada

Existem dois caminhos de entrada que convergem para o mesmo pipeline:

1. **Standalone:** `main.py` → `app/speech_ai_app.py::SpeechAIApp.run()` — usado para execução manual/local.
2. **Integração TAPOS:** `integration/cli.py` → `integration/facade.py::run_speech_ai()` → `integration/runner.py::execute_current_pipeline()` — usado pelo backend SaaS via subprocesso (`platform/saas-backend/app/products/speech_ai_adapter.py`).

Ambos chamam exatamente a mesma sequência: `TextAnalyzer.run()` → `SpeechIntelligenceEngine.build_profile()` → `NarrationBuilder` → `SpeechBuilder` → `SpeechService.synthesize()` — mas a lógica está duplicada entre `app/speech_ai_app.py` e `integration/runner.py`, não compartilhada em uma única função.

Duplicação encontrada: dois diretórios `integration/`

Existem **dois** diretórios de integração: `products/speech-ai/integration/` (raiz do produto: `facade.py`, `runner.py`, `schemas.py`, `cli.py`) e `products/speech-ai/app/integration/` (`facade.py`, `runner.py`, `schemas.py` — sem `cli.py`). Os dois são quase idênticos (o de `app/` tem pequenas diferenças de docstring e de cálculo de `project_root`). Confirmado em `platform/saas-backend/app/products/speech_ai_adapter.py` que **apenas o `integration/` de**

nível raiz está conectado à TAPOS — `app/integration/` é código morto, resquício de uma refatoração que moveu a integração para fora de `app/`. Deve ser removido em uma limpeza futura.

Diagrama de dependências (resumo)

```
TAPOS backend (adapter)
→ subprocess: .venv do speech-ai + integration/cli.py
→ integration/facade.py
  → integration/runner.py
    → pipeline/text_analyzer.py      (Presentation)
    → services/speech_intelligence.py (SpeechProfile)
    → pipeline/narration_builder.py  (narration.txt)
    → pipeline/speech_builder.py     (speech.txt)
    → services/speech_service.py
      → providers/provider_factory.py → providers/edge_provider.py
```

Ver também

- [pipeline.md](#) — detalhamento passo a passo do fluxo de processamento
- [components.md](#) — inventário de cada módulo
- [execution-engine.md](#) — como este pipeline é acionado de forma síncrona e assíncrona pela TAPOS

Pipeline

O pipeline de produção (idêntico nos dois pontos de entrada — ver [architecture.md](#)) segue cinco etapas:

1. Texto de entrada — análise

`pipeline/text_analyzer.py::TextAnalyzer`:- `load()` — lê o arquivo definido em `cfg.script_file`. - `normalize()` — limpeza de espaços em branco via regex. - `detect_slides()` — divide o texto por ocorrências de `slide\s+\d+` (regex). - `parse_slide()` — constrói `Slide` → `Paragraph` / `ListBlock` para cada bloco. - `calculate_statistics()` — gera `Statistics` (palavras, sentenças, parágrafos, caracteres, minutos estimados). - `run()` retorna um `Presentation` completo.

2. Speech Intelligence

`services/speech_intelligence.py::SpeechIntelligenceEngine.build_profile(presentation)` encadeia:

1. `LanguageDetector.detect()` → `Language` (pt/en/es/fr)
2. `VoiceSelector.select()` → perfil de voz padrão para o idioma detectado
3. `SpeechAnalyzer.analyze()` → `SpeechAnalysis` (métricas linguísticas)
4. `TimingCalculator.calculate()` → ritmo de leitura recomendado (WPM)
5. `SpeechOptimizer.optimize()` → `SpeechProfile` final (rate/pitch/pacing/confidence)

Ver [components.md](#) para o detalhe de cada serviço.

3. Construção da narração

`pipeline/narration_builder.py::NarrationBuilder` transforma o `Presentation` em texto de narração natural (quebra sentenças longas, insere pausas "..."), exportado como `narration.txt`.

4. Construção do texto de fala

`pipeline/speech_builder.py::SpeechBuilder` limpa e naturaliza o texto por idioma, produzindo o texto pronto para TTS, exportado como `speech.txt`.

5. Síntese

`services/speech_service.py::SpeechService.synthesize()`:- lê o arquivo de texto de fala; - chama `providers/provider_factory.py::ProviderFactory.create_from_speech_profile(speech_profile)` para

obter o provedor de TTS configurado; - envolve o provedor em `services/tts_engine.py::TTSEngine`; - chama `.generate()`, que produz o arquivo de áudio final (`audio.mp3`).

Pipeline experimental paralelo (não conectado à produção)

Existe um segundo pipeline, mais avançado, em `services/text_engines/`: Lexical → Clause → SpeechBlock → PausePlanner → SSML, capaz de gerar SSML real (`<speak>`, `<prosody>`, `<break>`). Ele **não está conectado ao fluxo de produção** — só é exercitado por um script manual de teste (`input/teste.py`). É a semente de um futuro pipeline de SSML real, atualmente fora do caminho síncrono/assíncrono usado pela TAPOS. Ver [roadmap.md](#).

Ver também

- [components.md](#) — inventário de módulos
- [models.md](#) — as estruturas de dados que atravessam cada etapa
- [outputs.md](#) — os artefatos gerados ao final do pipeline

Componentes

Pipeline (`pipeline/`)

Módulo	Responsabilidade
<code>text_analyzer.py::TextAnalyzer</code>	Texto bruto → <code>Presentation</code> (slides/parágrafos/ listas/estatísticas)
<code>narration_builder.py::NarrationBuilder</code>	<code>Presentation</code> → texto de narração natural
<code>speech_builder.py::SpeechBuilder</code>	<code>Presentation</code> → texto limpo, pronto para TTS

Serviços (`services/`)

Módulo	Responsabilidade
<code>speech_intelligence.py::SpeechIntelligenceEngine</code>	Orquestra os 5 sub-passos de inteligência em um <code>SpeechProfile</code>
<code>language_detector.py::LanguageDetector</code>	Detecção de idioma offline (<code>langdetect</code>), determinística (seed=0), cache por hash de texto, pt/en/es/fr
<code>voice_selector.py::VoiceSelector</code>	Escolhe o <code>VoiceProfile</code> padrão para um <code>Language</code> + provedor configurado, via <code>VoiceManager</code>
<code>voice_manager.py::VoiceManager</code>	Repositório somente-leitura de <code>VoiceProfile</code> s, carregado de <code>config/voices.json</code>
<code>speech_analyzer.py::SpeechAnalyzer</code>	Análise linguística baseada em estatística/ regex (métricas de palavra/sentença, diversidade lexical, heurísticas de complexidade/estilo/ritmo) → <code>SpeechAnalysis</code>
<code>timing_calculator.py::TimingCalculator</code>	Mapeia faixas de complexidade de sentença para WPM recomendado (com ajuste de -5 WPM para <code>pt</code>)
<code>speech_optimizer.py::SpeechOptimizer</code>	Converte <code>SpeechAnalysis</code> + <code>VoiceProfile</code> em <code>SpeechProfile</code> final; contém o hook no-op <code>_apply_future_ai_rules()</code> , reservado para regras de IA futura
<code>text_optimizer.py::TextOptimizer</code>	Regras de divisão/limpeza de sentença usadas por <code>SpeechOptimizer.optimize_text()</code>

Módulo	Responsabilidade
	— este método não parece ser chamado pelo pipeline de produção atual
<code>speech_service.py::SpeechService</code>	Orquestrador de topo: arquivo de narração + <code>SpeechProfile</code> → provedor → arquivo de áudio
<code>tts_engine.py::TTSEngine</code>	Adaptador fino que chama <code>provider.generate()</code> , desacoplando <code>SpeechService</code> de classes concretas de provedor

`services/text_engines/` — pipeline experimental (não conectado à produção)

Módulo	Responsabilidade
<code>lexical_analyzer.py::LexicalAnalyzer</code>	Tokenizador regex (Token/TokenType) — "base para IA futura", sem dependências externas
<code>clause_analyzer.py::ClauseAnalyzer</code>	Tokens → objetos <code>Clause</code> (MAIN/SUBORDINATE/etc.) com estimativas de pausa/confiança
<code>sentence_engine.py::SentenceEngine</code>	Divisor alternativo de sentenças longas (heurísticas de palavra/vírgula/conector)
<code>speech_block_builder.py::SpeechBlockBuilder</code>	Agrupar <code>Clause</code> s em blocos de fala naturais
<code>pause_planner.py::PausePlanner</code>	Atribui <code>estimated_pause_ms</code> por cláusula, conforme tipo/complexidade/posição
<code>ssml_engine.py::SSMLEngine</code>	Renderiza <code>Clause</code> s em SSML real (<code><speak></code> , <code><prosody></code> , <code><break></code>)

Provedores (`providers/`)

Ver [providers.md](#) para o detalhamento completo da abstração.

Modelos (`models/`)

Ver [models.md](#).

Configuração (`config/`)

Ver [configuration.md](#).

Ver também

- [architecture.md](#) — como estes componentes se conectam
- [pipeline.md](#) — a ordem de execução real

Provedores

A abstração

O Speech-AI foi construído para ser independente de fornecedor de síntese de voz, através de uma camada de abstração de provedores:

- `providers/base_provider.py::BaseTTSPProvider` (classe abstrata) — construtor recebe `voice, rate, pitch, volume`; método abstrato único: `generate(text, output_path) -> Path`.
- `providers/edge_provider.py::EdgeProvider(BaseTTSPProvider)` — única implementação real hoje. Usa `edge_tts.Communicate(...).save()` (assíncrono, encapsulado via `asyncio.run`). Expõe um `ProviderInfo` (`providers/provider_info.py`, dataclass congelada) com flags de capacidade: `supports_ssml=False`, `supports_streaming=False`, `supports_neural=True`, idiomas suportados (pt-BR, en-US, es-ES, fr-FR, de-DE, it-IT, ja-JP), formatos (mp3), `max_characters=100000`.
- `providers/provider_registry.py::ProviderRegistry` — dicionário `{provider_id: ProviderClass}` no nível de classe, com `register/get/exists/list/clear`.
- `providers/provider_loader.py::ProviderLoader.load()` — **auto-descoberta**: varre `providers/*_provider.py` (ignorando `base_provider.py`), importa cada módulo, localiza qualquer subclasse de `BaseTTSPProvider` e registra-a com um `provider_id` derivado do nome da classe (remove "Provider", minúsculas — ex.: `EdgeProvider` → `"edge"`). Disparado uma vez, na importação, via `providers/__init__.py`.
- `providers/provider_factory.py::ProviderFactory` — três construtores estáticos (`create`, `create_from_profile`, `create_from_speech_profile` — este último é o usado em produção), todos resolvendo via `ProviderRegistry.get(provider_id)` e instanciando com `voice/rate/pitch/volume`.

Como adicionar um novo provedor

Basta criar um arquivo `providers/azure_provider.py` definindo `class AzureProvider(BaseTTSPProvider)` com `generate()` implementado — ele é **automaticamente registrado** como `"azure"`, sem nenhuma outra mudança de código. Depois, basta configurar `"tts": {"provider": "azure", ...}` em `config/settings.json` e adicionar entradas correspondentes com `"provider": "azure"` em `config/voices.json`. Este desenho plugável está confirmado tanto no código quanto na documentação interna do produto (`docs/SpeechAI_Docs/Sprint/ADR.md`, ADR-003; `docs/SpeechAI_Docs/Foundation/ARCHITECTURE.md`).

Roadmap de provedores

A documentação interna do produto (`ROADMAP.md` , síntese técnica) nomeia consistentemente **Azure Cognitive Services** e **OpenAI TTS** como próximos provedores (meta interna "Q3 2026"), seguidos por AWS Polly e ElevenLabs mais adiante. Nenhum destes está implementado ainda — apenas o Edge TTS está em produção hoje.

Ver também

- [architecture.md](#)
- [local-ai.md](#) — provedores de voz não devem ser confundidos com a camada de IA local planejada para análise de conteúdo
- [roadmap.md](#)

IA Local

Estado atual: nenhum modelo de IA em execução

Ao contrário do Edital-AI, o Speech-AI **não executa hoje nenhum modelo de IA local ou em nuvem** (nenhum LLM, SLM ou embedding). Tudo o que poderia ser confundido com "IA" no produto é, na realidade, determinístico:

- `services/language_detector.py` usa a biblioteca `langdetect` — um classificador estatístico simples (n-gramas), não um modelo neural, com seed fixa (`DetectorFactory.seed = 0`) para determinismo.
- Toda a "inteligência" de conteúdo (pontuação de complexidade, ritmo de leitura, seleção de tom/velocidade, planejamento de pausas) é baseada em regras e regex determinísticas: `speech_analyzer.py`, `timing_calculator.py`, `speech_optimizer.py`, e todo o pacote `services/text_engines/`. Não há inferência de machine learning em nenhum ponto do pipeline de produção.

Um plano concreto e documentado para IA local

Existe, porém, um plano detalhado e específico para IA local no Speech-AI, descrito em `docs/Sprint 9 (Integracao com IA).docx` e `docs/SpeechAI Platform.docx` — uma visão de "Sprint 9 – Local AI Intelligence Platform" que rejeita explicitamente LLMs em nuvem ("Você NÃO deve usar LLM cloud") em favor de uma estratégia **local-first com modelos pequenos (SLM)**:

- **Runtime:** Ollama (ou llama.cpp), rodando localmente — consistente com a filosofia de IA local já usada pelo Edital-AI.
- **Modelos propostos:** Phi-3 mini (classificação de emoção/intenção, leve e rápido), Mistral 7B (fallback de NLP), LLaMA 3 8B (raciocínio mais pesado, opcional/futuro), Mixtral (alternativa mencionada).
- **Arquitetura em 3 camadas proposta:** 1. Camada 1 = motor de regras determinístico já existente (já construído, é o pipeline atual). 2. Camada 2 = SLM local (emoção/intenção/sumarização leve). 3. Camada 3 = modelo local mais pesado, opcional.
- **Módulos futuros nomeados, ainda inexistentes no código:** `slm_gateway.py`, `emotion_engine.py`, um "Intent Engine", "Semantic Analyzer" e "Speech Advisor". Confirmado por busca no repositório: nenhum arquivo relacionado a `slm/emotion/intent` existe hoje — esta é uma etapa apenas de planejamento.
- Existe um plano de sprint nomeado (9.1 Infraestrutura de IA Local → 9.7 Pipeline de Fine-tuning) em `SpeechAI Platform.docx`.

Pontos de extensão já reservados no código

Dois sinais concretos de que essa camada foi antecipada no design, mesmo sem implementação:

- `services/speech_optimizer.py::SpeechOptimizer._apply_future_ai_rules()` — um hook no-op real, já presente no código, com docstring listando "Azure OpenAI, OpenAI GPT, Claude, Gemini, Ollama, Emotion Engine, Storytelling Engine".
- `.env` do produto tem uma seção "FUTURE LOCAL AI" com chaves `AI_PROVIDER`, `AI_MODEL`, `AI_ENABLED` — presentes mas **não lidas por nenhum código** hoje (nenhum `os.getenv` referencia esses nomes).

Conclusão

IA local é parte deliberada e de primeira classe do roadmap do próprio Speech-AI — não é "assunto do Edital-AI" por engano. Simplesmente ainda não foi implementada: tudo o que está em produção hoje é NLP determinístico.

Ver também

- [roadmap.md](#) — onde esta camada se encaixa nos próximos passos
- [../03-products/edital-ai.md](#) — o produto onde IA local (Ollama) já está em produção
- [../00-tecle/vision.md](#) — a filosofia de "IA local sempre que possível" que motiva este plano

Motor de Execução

O Speech-AI roda nos dois modelos de execução suportados pela TAPOS: síncrono (resposta imediata) e assíncrono (fila + worker dedicado). Os dois caminhos convergem exatamente no mesmo pipeline interno (`integration/cli.py` → `facade.py` → `runner.py`) — a única diferença é se a requisição HTTP espera a resposta inline ou se um worker processa o pedido em segundo plano.

Caminho síncrono

```
POST /products/speech-ai/run (ou /upload)
↓
platform/saas-backend/app/products/speech_ai_adapter.py::run_speech_ai_product()
↓
subprocess: <speech-ai>/.venv/bin/python integration/cli.py
↓
integration/cli.py – adiciona PROJECT_ROOT ao sys.path
↓
integration/facade.py::run_speech_ai()
↓
integration/runner.py::execute_current_pipeline() – pipeline completo, síncrono
↓
SpeechRunResult (dataclass em integration/schemas.py: status, input_file,
narration_file, speech_file, audio_file, provider, voice, language,
estimated_minutes) → JSON em stdout → parseado pelo adapter FastAPI
```

Caminho assíncrono

```
POST /products/speech-ai/submit
↓
_submit_job() cria um registro Job (status="queued")
↓
publish_job(job_id, "speech-ai") (app/jobs/publisher.py)
↓
RabbitMQ – fila durável "speech_ai_jobs"
↓
Worker dedicado: platform/tap-runtime/workers/speech-ai-worker/worker.py
(basic_qos(prefetch_count=1))
↓
Para cada mensagem: marca job "running" no Postgres →
chama a MESMA run_speech_ai_product() do caminho síncrono →
marca job "completed"/"failed" com result_json/error_message
```

PID e log do worker em `.runtime/pids/speech-ai-worker.pid` e `.runtime/logs/speech-ai-worker.log`.

Nota sobre documentação desatualizada

O `README.md` do produto descreve, em uma seção "Próxima Evolução — Task 015", a execução assíncrona via RabbitMQ/worker como trabalho **futuro**. Na realidade, essa capacidade **já está implementada** no código atual (worker, publisher, rotas `/submit`) — o README está desatualizado em relação ao estado real da plataforma. Trate o código (`platform/tap-runtime/workers/speech-ai-worker/`, `platform/saas-backend/app/jobs/publisher.py`) como fonte de verdade, não o README.

Ver também

- [../04-platform/gateway.md](#) — o gateway central que autoriza estas chamadas
- [../04-platform/runtime.md](#) — arquitetura de fila e workers no nível da plataforma
- [architecture.md](#)

Modelos de Dados

Todos em `models/`, majoritariamente `@dataclass` simples (alguns com `slots=True`, um `frozen=True`) — sem Pydantic.

Modelo	Descrição
<code>language.py::Language</code> (frozen, slots)	<code>code</code> , <code>locale</code> , <code>name</code> , <code>confidence</code> + propriedades <code>is_portuguese/is_english/is_spanish/is_french</code> . Produzido por <code>LanguageDetector</code> .
<code>voice_profile.py::VoiceProfile</code> (slots)	Configuração completa de voz TTS: <code>profile_id</code> , <code>provider</code> , <code>language</code> , <code>locale</code> , <code>voice</code> , <code>name</code> , <code>description</code> , <code>rate</code> , <code>pitch</code> , <code>volume</code> , <code>style</code> , <code>role</code> , <code>gender</code> , <code>is_default</code> . Carregado de <code>config/voices.json</code> por <code>VoiceManager</code> .
<code>speech_profile.py::SpeechProfile</code>	O "contrato" final para síntese: <code>language</code> , <code>voice</code> , <code>locale</code> , <code>provider</code> , <code>rate</code> , <code>pitch</code> , <code>volume</code> , <code>recommended_wpm</code> , <code>reading_style</code> , <code>pacing</code> , <code>complexity</code> , <code>confidence</code> , <code>estimated_minutes</code> . Produzido por <code>SpeechOptimizer</code> , consumido por <code>SpeechService / ProviderFactory</code> .
<code>speech_analysis.py::SpeechAnalysis</code>	Resultado intermediário de análise linguística (palavras/sentenças/parágrafos/complexidade/estilo de leitura/WPM recomendado etc.), produzido por <code>SpeechAnalyzer</code> , refinado por <code>TimingCalculator</code> .
<code>presentation.py::Presentation</code> (slots)	Container de topo do documento: <code>title</code> , <code>slides: list[Slide]</code> , <code>statistics: Statistics</code> ; métodos <code>add_slide</code> , <code>to_text()</code> (achata tudo para detecção de idioma/análise), <code>summary()</code> .
<code>slide.py::Slide</code>	<code>number</code> , <code>title</code> , <code>paragraphs: list[Paragraph]</code> , <code>lists: list[ListBlock]</code> ; helpers <code>add_paragraph / add_list</code> .
<code>paragraph.py::Paragraph</code>	<code>text</code> , <code>slide_index</code> , <code>importance</code> (1=normal/2=importante/3=destaque — este campo não é usado em nenhuma lógica atual do pipeline; provável reserva para ênfase futura em SSML).
<code>list_block.py::ListBlock</code>	<code>items: list[str]</code> , <code>slide_index</code> .
<code>statistics.py::Statistics</code> (slots)	<code>characters</code> , <code>words</code> , <code>sentences</code> , <code>paragraphs</code> , <code>estimated_minutes</code> + propriedades derivadas (<code>estimated_seconds</code> , <code>words_per_sentence</code> , <code>characters_per_word</code>).
<code>clause.py::Clause</code> (slots) + <code>ClauseType</code>	Enum MAIN/SUBORDINATE/ENUMERATION/INTRODUCTION/CONCLUSION/UNKNOWN — pertence ao pipeline experimental <code>text_engines/</code> , ainda não conectado à produção; docstring já lista consumidores

Modelo	Descrição
	futuros ("StorytellingEngine", "EmotionEngine") que ainda não existem no repositório.

Fluxo de transformação

Texto bruto → Presentation (Slide[Paragraph, ListBlock], Statistics)
 → Language (via LanguageDetector)
 → SpeechAnalysis (via SpeechAnalyzer)
 → SpeechProfile (via TimingCalculator + SpeechOptimizer, usando VoiceProfile)

Ver também

- [pipeline.md](#) — onde cada modelo é produzido/consumido
- [configuration.md](#) — origem dos dados de VoiceProfile

Saídas

Artefatos reais confirmados em `products/speech-ai/output/` :

Arquivo	Conteúdo
<code>narration.txt</code>	Texto de narração natural gerado pelo <code>NarrationBuilder</code>
<code>speech.txt</code>	Texto limpo pronto para TTS, gerado pelo <code>SpeechBuilder</code>
<code>speech.xml</code>	Apesar da extensão <code>.xml</code>, o conteúdo é texto plano de narração com marcadores literais de pausa <code>"..."</code> — não é XML/SSML real. Resquício de uma versão anterior do pipeline, anterior ao nome atual <code>speech.txt</code> em <code>settings.json</code> .
<code>speech_report.json</code>	Estatísticas da execução, ex.: <code>{"characters": 6851, "words": 974, "sentences": 90, "paragraphs": 95, "estimated_minutes": 6.72}</code> — corresponde ao formato de <code>Statistics.to_dict()</code>
<code>script_optimized.txt</code>	Saída de uma passada de otimização sobre o script de demonstração (inglês, cenário de governança de IA)
<code>audio.mp3</code>	Arquivo de áudio real gerado (~4 MB na última execução registrada)

Uma execução real recente processou um livro inteiro (biografia, +29 mil linhas) como entrada — prova de que o pipeline escala além de slides/scripts curtos para textos longos, sem limite de chunking aparente.

Subpastas reservadas, ainda vazias

`output/audio/`, `output/reports/`, `output/scripts/`, `output/ssml/` existem como estrutura, mas estão vazias hoje — scaffolding para uma organização de saída mais estruturada no futuro (por exemplo, arquivos de SSML real quando o pipeline `services/text_engines/ssml_engine.py` for conectado à produção).

Evidência de execução completa

`logs/app.log` (301 KB) registra uma execução real de ponta a ponta: Text Analyzer → Speech Intelligence → Narration/Speech builders → síntese via Edge TTS → "Audio generated successfully".

Ver também

- [pipeline.md](#) — como cada saída é produzida
- [roadmap.md](#) — o plano (ainda não implementado) de SSML real

Configuração

config/settings.json

- `input.script_file` — caminho do texto de origem
- `tts.provider` + `tts.voice_profile` — seleciona um perfil de `voices.json`
- `output.directory` / `output.filename`
- `speech.pause_short` / `pause_long` / `split_long_sentences`
- `project.name` / `project.version`

config/voices.json

- `defaults` — mapa idioma → `profile_id` (hoje: `en` → `en-us-guy`, `pt` → `pt-br-francisca`)
- `profiles` — 4 perfis definidos: `en-us-guy`, `en-us-jenny`, `pt-br-francisca`, `pt-br-antonio`, cada um com `provider/language/locale/voice/name/description/rate/pitch/volume/style/role/gender`

config/config_manager.py::ConfigManager

Carrega os dois arquivos JSON na construção e mantém um `VoiceManager` interno. Expõe:

- `provider`, `voice_profile_name`, `voice_profile`
- **atalhos de compatibilidade retroativa:** `voice`, `language`, `locale`, `rate`, `pitch`, `volume` — todos proxies para `voice_profile`. Um comentário no próprio código indica que **serão removidos na Sprint 9** (a mesma sprint planejada para a camada de IA local — ver [local-ai.md](#)).
- `script_file`, `output_directory` / `output_filename`
- `pause_short` / `pause_long` / `split_sentences`
- `project_name` / `project_version`
- `words_per_minute = 145` — hardcoded no código, apesar de parecer configurável via JSON; não é de fato lido do arquivo.

.env

Carregado via `python-dotenv` em `main.py`:

- `PROJECT_NAME`, `PROJECT_ENV`, `INPUT_FILE`, `OUTPUT_DIR`, `OUTPUT_AUDIO_FILE`, `LOG_LEVEL`, `TTS_PROVIDER`
- Seção "**FUTURE LOCAL AI**": `AI_PROVIDER`, `AI_MODEL`, `AI_ENABLED` — presentes no arquivo mas **não lidos por nenhum código hoje**. Apenas `PROJECT_ENV` é de fato utilizado (em `main.py`).

Ver também

- [local-ai.md](#) — o significado da seção "FUTURE LOCAL AI" no `.env`
- [providers.md](#) — como `tts.provider` se conecta à fábrica de provedores
- [models.md](#) — `VoiceProfile`, o modelo que espelha `voices.json`

Roadmap

Maturidade atual

O pipeline central — análise de texto → Speech Intelligence → construção de narração/fala → síntese via Edge TTS — é real, funcional, e evidenciado por logs e artefatos de execução (ver [outputs.md](#)). A integração com a TAPOS (rotas síncronas e assíncronas, adapter, publisher RabbitMQ, worker dedicado) também é real e está operacional — confirmado diretamente em `platform/saas-backend` e `platform/tap-runtime/workers/speech-ai-worker`.

O que está pronto, mas não conectado à produção

- **Pipeline experimental de SSML** (`services/text_engines/`: Lexical → Clause → SentenceEngine → SpeechBlockBuilder → PausePlanner → SSMLEngine) — código completo, produz SSML real, mas só é exercitado por um script manual (`input/teste.py`), fora do caminho de produção.
- `SpeechOptimizer._apply_future_ai_rules()` — hook no-op já presente no código, reservado para regras de IA (Azure OpenAI, GPT, Claude, Gemini, Ollama, Emotion Engine, Storytelling Engine).
- **Camada de IA local/SLM** (Ollama + Phi-3 mini/Mistral 7B para emoção/intenção) — planejamento detalhado em documentos internos ("Sprint 9"), zero código implementado (`slm_gateway.py`, `emotion_engine.py` não existem). Ver [local-ai.md](#).
- Apenas **um provedor de TTS** implementado (Edge); Azure, OpenAI TTS, AWS Polly e ElevenLabs são citados em múltiplos documentos como próximos passos, mas ausentes no código.
- `Paragraph.importance` — campo existente no modelo, não utilizado por nenhuma lógica do pipeline (provável reserva para ênfase futura em SSML).
- `services/text_optimizer.py::TextOptimizer` e `SpeechOptimizer.optimize_text()` não parecem ser chamados pelo pipeline de produção atual — possível código morto a confirmar.

Limpeza técnica pendente

- `products/speech-ai/app/integration/` é um duplicado obsoleto de `products/speech-ai/integration/` (o único conectado à TAPOS) — candidato a remoção.
- **Nenhuma suíte de testes automatizados existe** no produto (zero arquivos `test_*.py` / `conftest.py`), apesar de a documentação interna (`docs/SpeechAI_Docs/Developer/TESTING_GUIDE.md`) e materiais de síntese técnica citarem métricas como "99,2% de taxa de sucesso" e "87% de cobertura de testes" — esses números devem ser tratados como **conteúdo aspiracional/gerado**, não fato verificado (o próprio documento de origem se identifica como gerado por IA externa).

Documentação interna desatualizada em relação ao código

Vale registrar com transparência, para futuras revisões desta knowledge base:

- `docs/SpeechAI_Docs/Sprint/ROADMAP.md` lista API REST e processamento em lote via fila de mensagens como trabalho futuro ("Q1/Q2 2027") — mas ambos **já existem** no código atual (rotas FastAPI da TAPOS + worker RabbitMQ).
- A numeração de "Sprint" é inconsistente entre documentos internos: um changelog do produto chama o trabalho de Provider Factory de "Sprint 9", enquanto `SpeechAI Platform.docx` reserva "Sprint 9" especificamente para a (ainda não iniciada) Plataforma de IA Local.
- `README.md` do produto descreve a execução assíncrona (Task 015) como "próxima evolução" — já implementada.

Próximos passos recomendados

1. Conectar o pipeline de SSML (`text_engines/`) ao fluxo de produção, substituindo os marcadores de pausa em texto plano por SSML real.
2. Implementar a camada de IA local planejada (Ollama + SLM) para emoção/intenção, seguindo o plano de "Sprint 9" já documentado.
3. Adicionar um segundo provedor de TTS (Azure ou OpenAI) para validar de fato a abstração de provedores em produção.
4. Criar uma suíte de testes automatizados mínima antes de reivindicar qualquer métrica de cobertura.
5. Remover o diretório `app/integration/` duplicado.
6. Atualizar a documentação interna do produto para refletir o que já está implementado (rotas assíncronas, integração TAPOS).

Ver também

- [local-ai.md](#) — detalhamento do plano de IA local
- [providers.md](#) — roadmap específico de provedores de TTS
- [../01-tapos/roadmap.md](#) — lacunas no nível da plataforma

Produtos

Edital-AI

O que é

O Edital-AI automatiza a leitura e análise de editais de licitação pública brasileira. Um analista humano hoje lê um edital inteiro — frequentemente entre 50 e 300 páginas, em linguagem jurídica repetitiva, com tabelas de itens que se espalham por dezenas de páginas — para decidir se vale a pena participar, o que precisa ser entregue e até quando. O Edital-AI automatiza exatamente essa etapa.

O usuário faz upload de um edital em PDF, DOCX, TXT ou MD. O sistema extrai, de forma **determinística** (leitura de tabelas nativas do documento e reconhecimento de padrões — sem depender de IA para os dados críticos): número do edital, modalidade, órgão licitante, objeto da licitação, cada item e lote com quantidade e valor estimado, os documentos obrigatórios de habilitação, e todos os prazos críticos (sinalizando automaticamente quando resta menos de 7 dias para um prazo importante).

Sobre essa base estruturada, um modelo de IA rodando **localmente via Ollama** (modelo `mistral`) gera um resumo executivo, identifica riscos e oportunidades, e calcula um score de conformidade de 0 a 100. Se a camada de IA falhar ou estiver indisponível, um mecanismo de segurança determinístico garante que o pipeline nunca trava por causa da IA — confiabilidade antes de sofisticação.

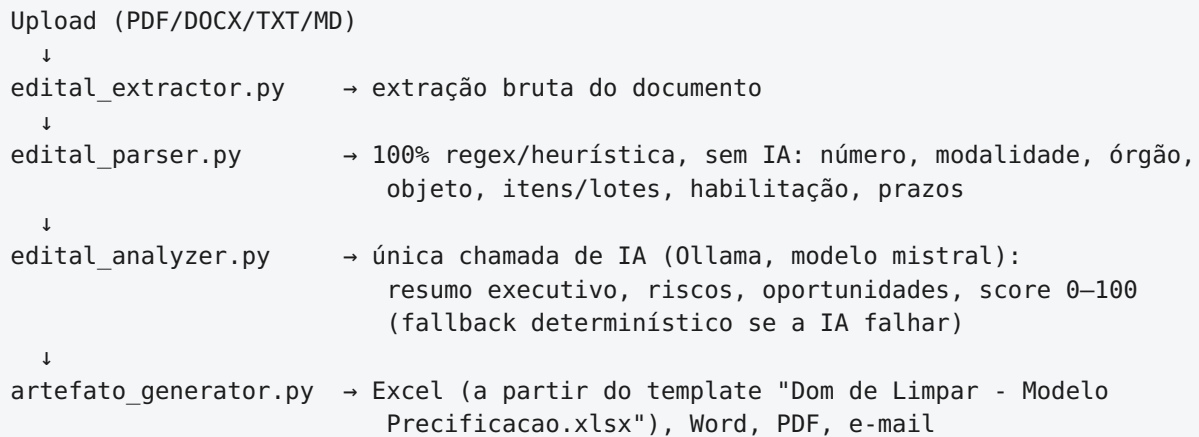
O resultado chega em quatro formatos prontos para uso: planilha Excel (5 abas), documento Word, PDF executivo e rascunho de e-mail já destacando os prazos mais urgentes.

Documentação detalhada

O produto já possui documentação técnica interna madura em `products/edital-ai/edital-ai-docs/`, que serve como fonte primária e não é reproduzida aqui na íntegra:

Tópico	Arquivo
Visão de negócio e casos de uso	<code>01-visao-geral-e-negocio.md</code>
Arquitetura em camadas, fluxo síncrono/assíncrono	<code>02-arquitetura.md</code>
Modelos de dados (dataclasses e schemas de banco)	<code>03-modelos-de-dados.md</code>
Pipeline função a função	<code>04-pipeline-e-funcoes.md</code>
Integração com o SaaS backend	<code>05-integracao-saas-backend.md</code>
Configuração e execução local	<code>06-configuracao-e-execucao.md</code>
Limitações e débito técnico	<code>07-limitacoes-e-debito-tecnico.md</code>
Briefing para comercialização	<code>08-briefing-para-comercializacao.md</code>
A chamada de IA em detalhe	<code>Chamada_IA_Edital.md</code>

Arquitetura e pipeline



Modelos de dados

Dataclasses internas: `Objeto`, `Requisito`, `Prazo`, `Editais / Secao`, `Analise`, `Artefato`, `EditaisRunResult`. Do lado da plataforma: `Job`, `EditaisAnalise` (tabelas de banco).
Detalhamento completo em `03-modelos-de-dados.md`.

Exemplo real

`output/Editais_CasaGrande_16072026.{pdf,docx,xlsx}` + `_email.txt` — Edital nº 014/2026, Município de Casa Grande - MG, pregão eletrônico, 62 itens, 30 requisitos, score de conformidade 100%. A pasta `processados/` arquiva 11 editais reais já processados (municípios de MGS, São Joaquim de Bicas, Fortuna Minas, Casa Grande, MGM Panos) — confirmando uso repetido com documentos reais, não apenas um teste isolado.

Integração com a TAPOS

Segue o mesmo padrão `facade/runner/schemas/cli` dos demais produtos (ver [../05-development/coding-standards.md](#)):

- `integration/facade.py::run_editais_ai`
- `integration/runner.py::execute_pipeline` → `EditaisRunResult`
- `integration/cli.py` — stdout reservado ao JSON final; exceções vão para stderr + exit code 1
- Adapter na plataforma: `platform/saas-backend/app/products/editais_ai_adapter.py`
- Worker dedicado: `platform/tap-runtime/workers/editais-ai-worker/worker.py`

Maturidade e limitações

Piloto validado em modo **single-tenant**: `input/ / output/` tratam apenas um edital "em foco" por vez em toda a instalação — este é o principal bloqueador para operação multi-cliente. Outros pontos de débito técnico conhecido (detalhados em `07-limitacoes-e-debito-tecnico.md`):

- caminhos em disco local, não armazenamento de objetos;
- `EditalRunResult` retorna apenas contagens, não a lista completa de itens/requisitos/prazos;
- a heurística de extração de tabelas já precisou de correção uma vez (um edital real produziu 0 itens por nomes de coluna não reconhecidos e tabela dividida em múltiplas páginas — corrigido, mas ainda heurístico, não garantido);
- `score_conformidade` mede completude de extração, não risco jurídico;
- sem limite de tamanho de arquivo ou timeout aplicado apesar de existirem campos para isso em `settings.json`;
- dependências não utilizadas (`jinja2` , `python-pptx`);
- vários campos de modelo nunca populados (`Edital.secoes` , `Objeto.modalidade_entrega / fabricante_sugerido` , `Requisito.prazo_validade / observacoes` , `Prazo.hora`).

Roadmap

Isolamento de dados por cliente, armazenamento de objetos com URLs assinadas, indicador de confiança por extração, extração assistida por IA como fallback, API mais rica expondo os dados estruturados completos, pipeline de deploy real, e decisão entre IA local ou provedor gerenciado — todos detalhados em `08-briefing-para-comercializacao.md`.

Ver também

- [code-ai.md](#) — pode se tornar camada de conversão de entrada reaproveitável pelo próprio Edital-AI no futuro
- [../01-tapos/roadmap.md](#) — lacunas de multi-tenancy no nível da plataforma

Educa-AI

Status: reservado para desenvolvimento futuro

O Educa-AI é a quarta vertical de IA planejada para a TAPOS. Diferente de Speech-AI, Edital-AI e Code-AI, **não existe implementação hoje**: `products/educa-ai/` é um diretório completamente vazio (sem arquivos, sem subpastas), criado como reserva de estrutura desde o início do workspace.

Não há adapter, worker, pasta `integration/` ou qualquer código associado a este produto em nenhum outro ponto da plataforma.

Por que ele existe mesmo sem código

A existência deste diretório vazio — e sua menção explícita em `architecture.md` e no documento para investidores-anjo — é, em si, parte da tese de plataforma da Tecle: a TAPOS foi desenhada para que uma nova vertical de IA seja adicionada como uma extensão sobre a infraestrutura já existente (autenticação, assinaturas, gateway, execução assíncrona), não como um novo projeto de engenharia começado do zero. O Educa-AI é a prova, no próprio código, de que essa reserva de espaço já foi planejada.

"É o motivo pelo qual já existe uma quarta vertical, educa-ai, reservada na estrutura da plataforma para os próximos ciclos de desenvolvimento." — [Documento para Investidores-Anjo](#)

Escopo descrito (não implementado)

Segundo `architecture.md`, o Educa-AI receberá dados e documentos financeiros e apoiará educação financeira e acompanhamento de mercado, utilizando a mesma base SaaS dos demais produtos. Não há, além deste parágrafo, nenhum detalhamento adicional de arquitetura, pipeline, modelos de dados ou roadmap técnico em nenhum lugar do repositório.

O que precisaria ser construído

Ao ser priorizado, o Educa-AI deverá seguir o mesmo padrão já validado pelos outros três produtos:

1. Estrutura de produto com `integration/{facade.py, runner.py, schemas.py, cli.py}` (ver [../05-development/coding-standards.md](#)).
2. Adapter dedicado em `platform/saas-backend/app/products/educa_ai_adapter.py`.
3. Worker dedicado em `platform/tap-runtime/workers/educa-ai-worker/`.

4. Registro do produto no modelo de Produtos e Assinaturas da TAPOS (task 011/012), com seu próprio slug.

Nenhum desses passos exige mudança na plataforma em si — é exatamente esse reaproveitamento sem reengenharia que a tese da TAPOS promete.

Ver também

- [edital-ai.md](#), [code-ai.md](#) — os três produtos hoje implementados, cujo padrão de integração o Educa-AI deverá seguir
- [../01-tapos/roadmap.md](#) — próximos passos da plataforma
- [../00-tecle/vision.md](#) — a tese de plataforma que justifica esta reserva

Code-AI

O que é

O Code-AI é um conversor universal de documentos corporativos para Markdown, pensado para consumo por LLMs e pipelines de RAG. Toda empresa que tenta adotar IA de forma séria esbarra no mesmo problema: seus dados mais importantes não estão em texto limpo — estão em PDFs, planilhas, apresentações, documentos Word e imagens escaneadas, formatos que modelos de linguagem não conseguem consumir de forma eficiente.

O Code-AI recebe documentos em PDF, DOCX/DOC, XLSX/XLS/CSV, PPTX/PPT, PNG/JPG/JPEG, TXT e MD, e produz uma versão padronizada, limpa e em Markdown. Para documentos digitalizados ou imagens, usa OCR para extrair o texto mesmo quando ele não existe em formato digital nativo.

O ganho não é apenas de compatibilidade — é de custo: cada consulta a um LLM é cobrada por token, e documentos corporativos brutos desperdiçam uma quantidade grande de tokens em ruído estrutural. A documentação do produto estima economia de 70–85% de tokens dependendo do formato de origem.

Nota de precisão: a única execução real capturada em `output/relatorio.json` mostra uma economia de 50,03% (348,08 KB → 173,94 KB) — abaixo da faixa de 70–85% citada na documentação de marketing do produto (docs/). Vale tratar 70–85% como estimativa por tipo de formato, não como resultado medido em todos os casos, até que mais execuções reais sejam registradas.

Arquitetura

Padrão *Strategy*: `src/conversor_markdown.py::ConvertorUniversal` despacha por extensão para uma classe conversora dedicada:

Formato	Conversor	Mecanismo
PDF	<code>ConversorPDF</code>	<code>pdfplumber</code> , com fallback para OCR via <code>pytesseract</code> + <code>pdf2image</code>
DOCX/DOC	<code>ConversorDocx</code>	<code>python-docx</code>
XLSX/XLS/CSV	<code>ConversorExcel</code>	<code>pandas</code> , lendo todas as abas
PPTX/PPT	<code>ConversorPowerPoint</code>	<code>python-pptx</code>
PNG/JPG/JPEG	<code>ConversorImagem</code>	OCR
TXT/MD		passthrough

Formato	Conversor	Mecanismo
	ConversorTexto / ConversorMarkdown	

`ConfiguradorDependencias` instala automaticamente dependências ausentes via `pip install --break-system-packages` no momento da importação — uma conveniência de desenvolvimento que **não é recomendada para produção** (ver Limitações).

Pontos de entrada: `main.py` (converte todo o diretório `input/`), `scripts/rapido.py` (CLI mínima), `scripts/workflow.py` (5 workflows nomeados, incluindo "otimizar" e "preparar para IA"), `scripts/installar.py` (validador/installador de ambiente), `scripts/exemplos_uso.py` (exemplos de uso). O produto em si não tem lógica de fila assíncrona própria — a execução assíncrona é inteiramente responsabilidade da camada TAPOS.

Modelos de dados

Não há dataclasses internas ao produto: `ConvertorUniversal.relatorio` é um dicionário com `data`, `arquivos_processados` (lista de `{arquivo_original, arquivo_convertido, tamanho_original_kb, tamanho_convertido_kb, economia_percentual}`) e `erros`. A camada de integração formaliza isso em `integration/schemas.py`: `ConvertedFile` (`input_file`, `output_file`, `size_input_kb`, `size_output_kb`, `savings_percent`) e `CodeAIRunResult` (`status`, `files_processed`, `converted`, `errors`, `report_file`).

Exemplo real

`input/documento.pdf` é um edital real (MGS — Minas Gerais Administração e Serviços S.A., Pregão Eletrônico Registro de Preços nº 029/2026, materiais de limpeza), convertido em `output/documento.md` (2403 linhas, Markdown página a página, tabelas renderizadas via `df.to_markdown()`), com o relatório de economia em `output/relatorio.json`.

Integração com a TAPOS

Mesmo padrão facade/runner/schemas/cli dos demais produtos, confirmado byte a byte idêntico ao do Edital-AI:

- `integration/facade.py::run_code_ai(input_file) → integration/runner.py::execute_current_pipeline → CodeAIRunResult.to_dict()`
- `integration/cli.py` redireciona os prints internos para um buffer e imprime só o JSON final em stdout; exceções vão para stderr + `sys.exit(1)`
- Adapter na plataforma: `platform/saas-backend/app/products/code_ai_adapter.py::run_code_ai_product()`, executando `<code-ai>/.venv/bin/python integration/cli.py <arquivo>` via subprocess
- Worker dedicado: `platform/tap-runtime/workers/code-ai-worker/worker.py`

Maturidade e limitações

Bem menos documentado que o Edital-AI — não existe ainda um equivalente ao débito técnico ou briefing de comercialização daquele produto. Pontos a observar:

- instalação automática de dependências via `pip` na importação é um risco operacional real em produção;
- sem streaming/progresso assíncrono, sem limite de tamanho de arquivo evidente;
- tratamento de erro por conversor apenas captura exceções e retorna uma string Markdown de erro ("# Erro ao converter X"), em vez de falhar de forma explícita.

Roadmap

Itens mencionados como aspiracionais em `docs/arquitetura_visual.md` ("Evolução Futura") e `docs/analise_projeto.md` ("Próximas Evoluções"), ainda não implementados: API REST própria, interface web, processamento assíncrono nativo do produto, cache, auto-vetorização, conectores SharePoint/OneDrive, pipeline de RAG.

Ver também

- [edital-ai.md](#) — potencial consumidor futuro do Code-AI como camada de conversão de entrada
- [../01-tapos/technologies.md](#) — stack compartilhado da plataforma

Platform

Runtime

Ambiente local de desenvolvimento

O ambiente é iniciado por `start_tapos_env.sh` (raiz do workspace), que:

1. valida que a estrutura esperada existe (diretório do backend, `.venv`, `worker.py`, `app/main.py`);
2. sobe `postgres` e `rabbitmq` via `docker compose up -d (/data/platform/infra/)`;
3. inicia o backend FastAPI (`uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload`) em background, com PID em `.runtime/pids/saas-backend.pid` e log em `.runtime/logs/saas-backend.log`, aguardando `/health` responder;
4. inicia o worker do Speech-AI (`platform/tap-runtime/workers/speech-ai-worker/worker.py`) da mesma forma, exportando `DATABASE_URL`, `WORKER_DATABASE_URL`, `RABBITMQ_URL` e `SAAS_BACKEND_ROOT`;
5. imprime URLs úteis (backend, `/docs`, `/openapi.json`, `/ui/`) e um comando `curl` de login pronto para uso.

Nota: apenas o worker do Speech-AI é iniciado automaticamente por este script hoje. Os workers de `editai-ai` e `code-ai` já existem em `platform/tap-runtime/workers/`, mas precisam ser iniciados manualmente — ainda não estão conectados ao bootstrap.

Containers ativos (verificado via `docker ps`)

Container	Imagem
<code>postgres</code>	<code>postgres:15</code>
<code>redis</code>	<code>redis:7-alpine</code>
<code>rabbitmq</code>	<code>rabbitmq:3-management</code>
<code>qdrant</code>	<code>qdrant/qdrant:latest</code>
<code>minio</code>	<code>minio/minio</code>
<code>ollama</code>	<code>ollama/ollama:latest</code>
<code>open-webui</code>	<code>ghcr.io/open-webui/open-webui:main</code>
<code>portainer</code>	<code>portainer/portainer-ce:latest</code>

O backend FastAPI e os workers **não são containerizados** — rodam como processos `uvicorn/python` diretos no host, gerenciados pelo script de bootstrap.

Isolamento por produto

Cada produto (`speech-ai` , `editai-ai` , `code-ai`) tem seu próprio `.venv` Python, completamente isolado do backend e dos demais produtos. A plataforma nunca importa código de produto diretamente — sempre invoca via subprocesso (ver [gateway.md](#)). Isso é real isolamento de processo, não apenas uma intenção de design.

Workers

```
platform/tap-runtime/workers/  
├─ speech-ai-worker/worker.py   → consome fila speech_ai_jobs  
├─ editai-ai-worker/worker.py   → consome fila editai_ai_jobs (prefetch_count=1)  
└─ code-ai-worker/worker.py     → consome fila code_ai_jobs
```

Cada worker é um processo Python standalone, dedicado a um único produto, consumindo sua própria fila RabbitMQ e atualizando o status do `Job` correspondente no PostgreSQL.

Diretórios de plataforma ainda vazios

`tap-os/` , `tap-platform/` , `tap-devops/` , `tap-ci/` , `tap-docs/` existem como namespaces reservados dentro de `platform/` , sem conteúdo implementado até o momento.

Ver também

- [storage.md](#) — onde os dados desses containers persistem
- [gateway.md](#) — como o gateway invoca workers e produtos isolados
- [../05-development/workspace.md](#) — estrutura completa do workspace de desenvolvimento

Storage

Princípio: dados nunca dentro de containers

Por regra de arquitetura (ver [Constituição](#)), toda persistência vive fora dos containers, em `/data/platform`, separado em `infra/`, `runtime/` e `storage/`. Essa regra foi verificada diretamente no sistema de arquivos, não é apenas uma intenção documentada:

```
/data/platform/storage/
├─ db/           → dados do PostgreSQL
├─ redis/        → dados do Redis
├─ rabbitmq/     → mnesia do RabbitMQ (filas, exchanges)
├─ minio/        → armazenamento de objetos (MinIO)
├─ qdrant/       → coleções vetoriais (inclui raft_state.json)
├─ models/       → modelos (inclui um par de chaves id_ed25519 e um diretório
cache/ – vale revisão de segurança sobre o que está versionado aqui)
└─ portainer/    → dados do Portainer
```

`/data`, `/runtime` e `/storage` estão excluídos do controle de versão via `.gitignore` — persistência nunca é versionada junto ao código.

Bancos e sistemas usados

Sistema	Papel atual
PostgreSQL	dados estruturados: usuários, produtos, assinaturas, jobs, análises de edital
Redis	reservado para cache/sessões — ainda não usado ativamente pelo backend
RabbitMQ	fila de mensagens para execução assíncrona (ver gateway.md)
MinIO	armazenamento de objetos — reservado; os produtos hoje gravam artefatos em disco local, não em MinIO (ver limitação em ../03-products/edital-ai.md)
Qdrant	banco vetorial — reservado para uso futuro de RAG
Ollama	modelos de IA local (usado hoje pelo <code>edital-ai</code> , modelo <code>mistral</code>)

Persistência ao nível de produto

Os três produtos ainda gravam entrada/saída em diretórios locais dentro de seu próprio caminho (`input/`, `output/`, `processados/`), não em MinIO — uma lacuna conhecida para operação multi-tenant (ver [../01-tapos/roadmap.md](#)).

Ver também

- [runtime.md](#) — os containers que produzem estes dados
- [../01-tapos/roadmap.md](#) — migração para object storage como próximo passo mapeado
- [../05-development/constitution.md](#) — o princípio de "persistência correta"

Segurança

Autenticação

`app/security.py` centraliza as duas primitivas de segurança da plataforma:

- **Hash de senha:** `passlib.context.CryptContext(schemes=["bcrypt"], deprecated="auto")`.
- **JWT:** `python-jose`, algoritmo `HS256`, com claim `sub` (email do usuário) e `exp` (expiração).

```
JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY", "dev-secret-key")
JWT_ALGORITHM = "HS256"
JWT_EXPIRE_MINUTES = int(os.getenv("JWT_EXPIRE_MINUTES", "60"))
```

Risco real a corrigir antes de produção: o segredo do JWT tem um valor padrão literal, `"dev-secret-key"`, aplicado sempre que a variável de ambiente `JWT_SECRET_KEY` não está definida. Isso é adequado para desenvolvimento local, mas é um risco de segurança concreto se a aplicação subir em produção sem essa variável explicitamente configurada.

Verificação de identidade

`app/deps.py::get_current_user` decodifica o token Bearer, extrai o `sub` (email), carrega o `User` correspondente no banco, e levanta `401 Unauthorized` em qualquer falha de validação (token ausente, inválido, expirado, ou usuário inexistente).

O que ainda não existe

Confirmado por leitura direta do código e cruzado com `architecture.md`:

- **refresh tokens** — sessão expira e exige novo login, sem renovação;
- **RBAC (papéis/perfis)** — não há diferenciação de papel de usuário, apenas identidade + assinatura por produto;
- **rate limiting** — nenhuma limitação de taxa de requisição implementada;
- **limite de tamanho/timeout de upload** — os endpoints de upload de `code-ai` / `edital-ai` aceitam arquivo sem validação de tamanho;
- **cofre de segredos** — segredos hoje vêm de variáveis de ambiente simples, sem integração com um vault dedicado;
- **multi-tenant** — não há isolamento de dados por cliente/organização, apenas por usuário individual.

O que já está implementado e validado

- hash de senha com bcrypt;
- autenticação via JWT em rota protegida;
- autorização por assinatura ativa antes de qualquer execução de produto (ver [gateway.md](#));
- persistência de dados fora de containers, em `/data/platform/storage` (ver [storage.md](#)).

Ver também

- [gateway.md](#) — como a autenticação se combina com a verificação de assinatura
- [api.md](#) — rotas de autenticação (`/auth/register` , `/auth/login`)
- [../01-tapos/roadmap.md](#) — hardening de segurança como lacuna mapeada para produção

Observabilidade

Estado atual: nenhuma observabilidade estruturada

Uma busca direta no código não encontrou métricas, tracing distribuído ou agregação de logs em nenhum ponto de `platform/`. O diretório `platform/tap-observability/` existe como namespace reservado, mas está vazio.

O que existe hoje é apenas:

- **logs em arquivo simples**, gravados pelo script de bootstrap: `.runtime/logs/saas-backend.log`, `.runtime/logs/speech-ai-worker.log`;
- **`print()` em workers**, sem formatação estruturada (ex.: mensagens como `"speech-ai-worker: aguardando jobs..."` impressas diretamente no processo);
- **critérios de aceite manuais** (`tasks/saas/*/acceptance.md`) com comandos `curl`, que funcionam como validação funcional pontual, não como observabilidade contínua.

O que não existe

- métricas (latência, taxa de erro, throughput de fila);
- tracing distribuído entre backend → adapter → subprocesso do produto → worker;
- dashboards ou alertas;
- logs estruturados (JSON) ou correlação de request-id entre camadas.

Por que isso importa

A ausência de observabilidade é uma lacuna conhecida e coerente com o estágio da plataforma (piloto validado, não operação multi-cliente em escala) — mas se torna prioritária assim que a plataforma precisar diagnosticar falhas de execução assíncrona (jobs presos em `running`, filas acumulando, workers travados) sem depender de inspeção manual de log.

Ver também

- [runtime.md](#) — os processos que hoje só geram logs de arquivo
- [gateway.md](#) — o fluxo assíncrono que mais se beneficiaria de tracing
- [../01-tapos/roadmap.md](#) — lacunas mapeadas para produção

API

O backend SaaS (`platform/saas-backend/app/`) é um único serviço FastAPI (`app/main.py`) que expõe todos os endpoints da plataforma. Não há um framework de gateway/API separado — a própria aplicação FastAPI, com autorização por rota, cumpre esse papel (ver [gateway.md](#)).

Endpoints existentes

Método	Rota	Arquivo	Propósito
GET	<code>/health</code>	<code>main.py</code>	healthcheck
POST	<code>/auth/register</code>	<code>routes/auth.py</code>	cria usuário, senha com hash bcrypt
POST	<code>/auth/login</code>	<code>routes/auth.py</code>	valida credenciais, retorna JWT
GET	<code>/users/me</code>	<code>routes/users.py</code>	perfil do usuário autenticado
GET	<code>/products</code>	<code>routes/products.py</code>	lista produtos ativos
GET	<code>/products/{slug}/access</code>	<code>routes/products.py</code>	verifica se o usuário tem assinatura ativa
POST	<code>/products/speech-ai/run</code>	<code>routes/products.py</code>	execução síncrona
POST	<code>/products/speech-ai/submit</code>	<code>routes/products.py</code>	execução assíncrona (enfileira job)
POST	<code>/products/speech-ai/upload</code>	<code>routes/products.py</code>	upload de roteiro + execução síncrona
POST	<code>/products/code-ai/{run,submit,upload}</code>	<code>routes/products.py</code>	mesmo padrão do speech-ai
POST	<code>/products/edital-ai/{run,submit,upload}</code>	<code>routes/products.py</code>	mesmo padrão, com validação extra de extensão e slot único de entrada
GET	<code>/products/edital-ai/historico</code>	<code>routes/products.py</code>	lista análises anteriores do usuário
GET	<code>/products/edital-ai/download/{analise_id}</code>	<code>routes/products.py</code>	download de artefato (excel , pdf , word , email)
POST	<code>/subscriptions</code>	<code>routes/subscriptions.py</code>	cria assinatura
GET	<code>/subscriptions</code>	<code>routes/subscriptions.py</code>	lista assinaturas do usuário
GET	<code>/jobs/{job_id}</code>	<code>jobs/routes.py</code>	status de um job assíncrono

Documentação automática do FastAPI disponível em `/docs` (Swagger) e `/openapi.json`. Um painel de teste estático (HTML puro, 1107 linhas) está montado em `/ui`, servido por `StaticFiles` a partir de `app/static/index.html` — não é uma interface de produto, é uma ferramenta de teste manual.

Padrão de rota

Todas as rotas de produto seguem o mesmo formato:

```
@router.post("/{produto}/run")
def run_produto(current_user: User = Depends(get_current_user), db: Session =
Depends(get_db)):
    _require_active_subscription(db, current_user, "produto-slug")
    result = run_produto_product()
    return result
```

Autenticação via `Depends(get_current_user)` (JWT), autorização via `_require_active_subscription` (verifica produto + assinatura ativa antes de qualquer execução) — ver [gateway.md](#) e [security.md](#).

Upload de arquivos

`code-ai` e `editai-ai` aceitam upload (`UploadFile`), validam a extensão contra uma lista permitida (`CODE_AI_ALLOWED_EXTENSIONS`, `EDITAI_AI_ALLOWED_EXTENSIONS`), e gravam o conteúdo em um diretório fixo de input do produto — sem limite de tamanho de arquivo aplicado no código atual.

Ver também

- [gateway.md](#) — o mecanismo de autorização compartilhado por todas as rotas de produto
- [security.md](#) — autenticação JWT/bcrypt usada por `get_current_user`
- [../05-development/coding-standards.md](#) — o padrão de código do backend FastAPI

Gateway

Não é um módulo separado — é um padrão repetido

Não existe uma classe ou serviço único chamado "Gateway" no código. O gateway de produtos é um **padrão aplicado consistentemente** em `platform/saas-backend/app/routes/products.py`: toda rota de produto chama `_require_active_subscription(db, current_user, product_slug)` antes de despachar qualquer execução.

```
def _require_active_subscription(db, current_user, product_slug) -> Product:
    product = db.query(Product).filter(Product.slug == product_slug).first()
    if not product:
        raise HTTPException(404, "Product not found")

    subscription = db.query(Subscription).filter(
        Subscription.user_id == current_user.id,
        Subscription.product_id == product.id,
        Subscription.is_active == True,
    ).first()

    if not subscription:
        raise HTTPException(403, "Access denied")

    return product
```

Nota de precisão: existe uma segunda implementação praticamente idêntica dessa mesma regra em `app/deps.py::get_product_access` (como uma `Depends` reutilizável), que não é usada pelas rotas atuais de `products.py` — as rotas reimplementam a checagem inline via `_require_active_subscription` em vez de reaproveitar `get_product_access`. É uma duplicação de lógica a resolver: consolidar em uma única implementação reduziria o risco de as duas regras divergirem no futuro.

Fluxo de autorização + despacho

```
Requisição → JWT válido? (get_current_user) → assinatura ativa?
(_require_active_subscription)
  ↓ sim para ambos
Adapter do produto (subprocess no .venv do produto) → resultado JSON
  ↓ não
403 Access denied / 401 Unauthorized
```

Nenhum produto é exposto diretamente ao usuário final — todo acesso passa por essa checagem dupla (identidade + assinatura) antes de qualquer execução, síncrona ou assíncrona.

Modo síncrono

A rota chama o adapter do produto diretamente (`run_speech_ai_product()`, `run_code_ai_product()`, `run_edital_ai_product()`), que executa `integration/cli.py` do produto como subprocesso no `.venv` isolado dele, e devolve o JSON de resultado na própria resposta HTTP.

Modo assíncrono

A rota `/submit` cria um registro `Job` (`status="queued"`) e chama `publish_job(job_id, product_slug)` (`app/jobs/publisher.py`), que publica em uma fila RabbitMQ dedicada por produto via `pika`:

```
QUEUE_NAMES = {
    "speech-ai": "speech_ai_jobs",
    "code-ai": "code_ai_jobs",
    "edital-ai": "edital_ai_jobs",
}
```

Cada fila é `durable=True` e a mensagem é publicada com `delivery_mode=2` (persistente). O worker dedicado do produto (`platform/tap-runtime/workers/{produto}-worker/worker.py`) consome a fila, executa o mesmo adapter/pipeline, e atualiza o `Job` (`status: queued` → `running` → `completed` / `failed` , com `result_json` ou `error_message`). O cliente consulta o resultado em `GET /jobs/{job_id}` (`jobs/routes.py`).

Isolamento por produto

Cada adapter (`app/products/{speech_ai,code_ai,edital_ai}_adapter.py`) invoca o `cli.py` do respectivo produto como subprocesso, usando o **.venv do próprio produto** — nunca o do backend. Se o produto quebrar, o processo do backend continua rodando; o gateway apenas recebe um `returncode` diferente de zero e levanta `RuntimeError` com `stdout/stderr` capturados.

Ver também

- [api.md](#) — todas as rotas que passam por este gateway
- [security.md](#) — a camada de autenticação que antecede a verificação de assinatura
- [../01-tapos/principles.md](#) — o princípio de desacoplamento que este mecanismo implementa
- [../05-development/coding-standards.md](#) — o padrão facade/runner/schemas/cli usado por todo adapter

Desenvolvimento

Workspace

Raiz única

Todo o desenvolvimento da Tecle acontece a partir de um único ponto de entrada:

```
/workspace/tecle
```

Esta é uma regra de engenharia, não apenas uma convenção: o Claude Code e qualquer desenvolvedor devem sempre iniciar a partir desta raiz, nunca abrindo subpastas isoladas no editor. Isso garante que o contexto de arquitetura e regras (`.claude/`) seja sempre carregado.

Estrutura real do workspace

```
/workspace/tecle
├── .claude/           # contexto persistente do Claude Code (ver claude-code.md)
├── .vscode/          # configuração mínima do editor
├── .runtime/         # PIDs e logs de processos rodando localmente (backend,
workers)
├── platform/         # backend SaaS e serviços de plataforma (TAPOS)
├── products/         # produtos SaaS independentes (speech-ai, edital-ai, code-
ai, educa-ai)
├── shared/           # SDKs, bibliotecas, templates, prompts, agentes –
reservado
├── governance/       # políticas, ADRs, arquitetura, padrões – reservado
├── developers/       # configuração de ambiente (VSCode, Claude, scripts) –
reservado
├── automation/       # Terraform, Ansible, scripts operacionais
├── tasks/            # tarefas de desenvolvimento documentadas (kernel/, saas/)
├── tests/            # reservado (os testes reais vivem em platform/saas-
backend/tests/)
├── playground/       # reservado, vazio
├── trash/            # descarte de artefatos de diagnóstico/depuração pontuais
├── investors/        # material para investidores
├── docs/             # esta knowledge base
├── architecture.md, changelog.md, readme.md # documentos de estado no nível raiz
└── start_tapos_env.sh # script de bootstrap do ambiente local
```

Nota de precisão: `.claude/workspace.md` documenta uma estrutura ligeiramente mais enxuta (sem `tasks/`, `trash/`, `.vscode/`, `.runtime/`). Esta seção reflete a estrutura real encontrada em disco, que é um superconjunto da documentada.

O que está implementado vs. reservado

Pasta	Estado
<code>platform/saas-backend/</code>	Implementado — FastAPI, testes próprios, rodando
<code>platform/tap-runtime/workers/</code>	Implementado — workers de speech-ai, edital-ai, code-ai
<code>products/speech-ai</code> , <code>products/edital-ai</code> , <code>products/code-ai</code>	Implementados — código real, integrados à plataforma
<code>products/educa-ai</code>	Reservado — diretório vazio, sem código
<code>tasks/saas/</code>	Implementado — 12 tarefas documentadas (005 a 015)
<code>tasks/kernel/</code>	Reservado — vazio
<code>shared/*</code> (agents, common, libraries, plugins, prompts, sdk-python, sdk-typescript, templates)	Reservado — todas as subpastas vazias
<code>governance/*</code> (adrs, architecture, constitution, policies, roadmap, standards)	Reservado — todas as subpastas vazias
<code>developers/*</code> (claude, extensions, scripts, snippets, vscode)	Reservado — todas as subpastas vazias
<code>tests/</code> , <code>playground/</code>	Reservado — vazios na raiz (os testes reais estão em <code>platform/saas-backend/tests/</code>)

Essa distinção importa: partes significativas da árvore de diretórios foram criadas antecipadamente, como scaffolding para fases futuras (multi-produto, SDKs compartilhados, governança formal), mas ainda não foram populadas. Ver [claude-code.md](#) para a "Expansion Strategy" que documenta essa intenção.

Runtime e persistência

A execução da plataforma (fora do workspace de desenvolvimento) acontece em `/data/platform`, separado em `infra/`, `runtime/` e `storage/` (banco, Redis, RabbitMQ, MinIO, Qdrant, modelos, Portainer). Por princípio de arquitetura, dados nunca vivem dentro de containers. `/data`, `/runtime` e `/storage` estão explicitamente excluídos do controle de versão via `.gitignore`.

Bootstrap do ambiente local

O script `start_tapos_env.sh` (raiz do workspace) automatiza a inicialização do backend SaaS e do worker do Speech-AI:

1. Valida que a estrutura esperada existe (diretório do backend, venv, `worker.py`, `app/main.py`).

2. Sobe `postgres` e `rabbitmq` via `docker compose up -d` (passo condicional a existir um `compose file` na raiz).
3. Inicia o backend FastAPI (`uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload`) em background, grava PID em `.runtime/pids/saas-backend.pid` e log em `.runtime/logs/saas-backend.log`, e faz polling em `/health` até responder.
4. Inicia o worker do speech-ai (`platform/tap-runtime/workers/speech-ai-worker/worker.py`) da mesma forma, exportando `DATABASE_URL`, `WORKER_DATABASE_URL`, `RABBITMQ_URL` e `SAAS_BACKEND_ROOT`.
5. Imprime as URLs úteis: backend (`http://localhost:8000`), Swagger (`/docs`), OpenAPI (`/openapi.json`), UI (`/ui/`) e um comando `curl` de login pronto para colar.

Nota: apenas o worker do speech-ai é iniciado por este script hoje. Os workers de edital-ai e code-ai já existem em `platform/tap-runtime/workers/`, mas ainda não estão conectados ao bootstrap automático — precisam ser iniciados manualmente.

Ver também

- [claude-code.md](#) — como o Claude Code usa este workspace
- [development-flow.md](#) — o ciclo de trabalho sobre esta estrutura
- [../01-tapos/architecture.md](#) — arquitetura técnica da plataforma que roda sobre este workspace

Constituição

Esta constituição é a fonte real de regras fundamentais de engenharia da TAPOS. Vive em `.claude/constitution.md` e é carregada automaticamente pelo Claude Code sempre que uma sessão é iniciada a partir de `/workspace/tecle`.

Nota de precisão: `governance/constitution/` existe como diretório reservado para uma futura constituição formal de governança, mas está **vazio** — não espelha nem substitui este documento. Enquanto isso não muda, `.claude/constitution.md` é a única constituição em vigor. O mesmo vale para `governance/architecture/`, também vazio; a arquitetura de estado atual real está documentada em `/workspace/tecle/architecture.md` e em [01-tapos/architecture.md](#).

Princípios fundamentais

1. Modularidade

Cada componente — backend SaaS, produtos, infraestrutura — deve ser independente e não depender diretamente dos outros.

2. Desacoplamento

- Produtos **nunca** acessam o banco de dados diretamente.
- Toda comunicação acontece **sempre** via API (ou, no caso dos produtos, via o contrato `facade/runner/schemas/cli` — ver [coding-standards.md](#)).
- Infraestrutura não é exposta ao domínio de negócio.

3. Simplicidade

Evitar complexidade desnecessária; implementar apenas o necessário; não antecipar problemas futuros sem evidência.

4. Incrementalidade

Desenvolvimento sempre em pequenos passos: um módulo por vez, testar antes de avançar, commit constante.

5. Testabilidade

Todo código deve poder ser executado isoladamente, ser validável, e não depender de contexto implícito.

6. Reprodutibilidade

O ambiente deve poder ser reconstruído do zero: infraestrutura declarativa, versionamento completo, sem dependências ocultas.

7. Persistência correta

Dados nunca dentro de containers — sempre em `/data/platform/storage`.

Regras de arquitetura

```
User
↓
SaaS Backend
↓
Products
↓
Infrastructure
```

Práticas proibidas

- Acesso direto ao banco por produtos.
- Lógica de negócio dentro da infraestrutura.
- Hardcode de configurações críticas.
- Implementar múltiplas features sem validação.

Padrão de engenharia por tarefa

```
Definir
↓
Implementar (1 módulo)
↓
Testar
↓
Commit
↓
Próximo
```

Objetivo

Construir uma plataforma escalável, modular, previsível e sustentável.

Honestidade sobre a aderência real

Esta constituição descreve o padrão pretendido. Vale registrar, com transparência, onde a prática observada até agora diverge dele — ver [development-flow.md](#) para o caso mais notável: o princípio de "commit constante" foi seguido no *planejamento* de cada tarefa (ver `tasks/saas/`), mas a execução real, até a baseline v1.0.0, resultou em um único commit acumulado (`a48b7b5 TAPOS v1.0.0 - operational SaaS baseline`), não em commits incrementais por módulo. Isso não invalida o princípio — reforça a necessidade de aplicá-lo de forma mais disciplinada daqui em diante.

Ver também

- [coding-standards.md](#) — como estes princípios se manifestam em código real
- [development-flow.md](#) — o ciclo de trabalho na prática
- [../01-tapos/principles.md](#) — a mesma constituição, aplicada à arquitetura da plataforma

Padrões de Código

Não existe hoje um guia de estilo formal e único (nenhum arquivo `STYLEGUIDE.md` ou equivalente). Os padrões abaixo foram observados diretamente no código real e devem ser tratados como a convenção vigente até que sejam formalizados em `governance/standards/` (hoje vazio).

Formatação

`.editorconfig` (raiz) define, para todos os arquivos:

```
root = true
[*]
indent_style = space
indent_size = 4
```

Nenhuma regra adicional de charset, final de linha ou espaço em branco está declarada — apenas indentação por espaços, 4 posições.

O padrão de integração de produtos: facade / runner / schemas / cli

Todo produto (Speech-AI, Edital-AI, Code-AI) expõe exatamente a mesma estrutura de integração em `integration/`, e a plataforma consome cada produto sempre da mesma forma. Este é o padrão de código mais importante do repositório, porque é o mecanismo concreto que garante o desacoplamento exigido pela [constituição](#):

- **schemas.py** — um `@dataclass` de resultado (ex.: `SpeechRunResult`, `EditalRunResult`, `CodeAIRunResult`) com um método `to_dict()` (via `dataclasses.asdict`).
- **runner.py** — a orquestração real do pipeline (`execute_current_pipeline()` / `execute_pipeline()`), importando os módulos internos do produto (`config`, `pipeline`, `services`) e retornando o `dataclass` de resultado.
- **facade.py** — ponto de entrada público e enxuto (ex.: `run_speech_ai()`, `run_edital_ai()`, `run_code_ai()`), que chama o runner e retorna `.to_dict()`. É esta a costura que o backend SaaS chama.
- **cli.py** — script apenas com `stdlib`, adiciona a raiz do projeto ao `sys.path`, chama a facade, e:
 - em sucesso: `print(json.dumps(result))` — só a saída final vai para `stdout`;
 - em falha: `traceback.print_exc(file=sys.stderr)` seguido de `sys.exit(1)`.

Do lado do backend, cada produto tem um *adapter* espelhado (`platform/saas-backend/app/products/{speech_ai,edital_ai,code_ai}_adapter.py`) que executa o `cli.py` do produto como subprocesso, usando o **.venv do próprio produto** (não o do backend):

```
subprocess.run([python_bin, cli_file], cwd=product_root, capture_output=True,
text=True)
```

O adapter verifica `returncode`, levanta `RuntimeError` com `stdout/stderr` capturados em caso de falha, e faz `json.loads(stdout)` em caso de sucesso. Este contrato — JSON de entrada/saída sobre um subprocesso isolado — é literalmente o "produtos nunca acessam o banco diretamente, comunicação sempre via API" da constituição, aplicado no nível de processo.

Estilo do backend FastAPI

Visto em `platform/saas-backend/app/routes/auth.py` e `app/security.py`:

- Rotas como funções simples com `APIRouter(prefix=..., tags=[...])`.
- Schemas de request definidos como `Pydantic BaseModel` diretamente no arquivo de rota (sem pasta `schemas/` separada no backend).
- Injeção de sessão via `Depends(get_db)`.
- Consultas diretas com `SQLAlchemy (.query().filter().first())`, sem camada de repositório.
- Erros via `HTTPException(status_code, detail)`.
- `security.py` centraliza hashing (`passlib.CryptContext(schemes=["bcrypt"])`) e JWT (`python-jose`), com segredo e expiração configuráveis por variável de ambiente (defaults de desenvolvimento: `dev-secret-key`, 60 minutos).
- Arquivos pequenos e diretos (tipicamente 25–90 linhas) — sem camada de serviço, sem abstrações extras.

Disciplina de escopo por tarefa

Os arquivos em `tasks/saas/*/task.md` reforçam repetidamente: "um módulo por vez", "evitar abstração desnecessária", "não antecipar problemas futuros sem evidência". Isso aparece na prática — por exemplo, a tarefa `007-login` proíbe explicitamente adicionar JWT naquele mesmo passo, deixando-o para a tarefa `008-jwt`. Esse escopo estreito e sequencial é, na prática, o padrão de código mais importante do repositório: cada módulo faz uma coisa, e a próxima camada só é adicionada quando a anterior está validada.

Ver também

- [development-flow.md](#) — como esse escopo por tarefa se conecta ao ciclo de commits
- [../04-platform/gateway.md](#) — o gateway que consome os adapters descritos acima

Claude Code

Propósito da pasta `.claude/`

A pasta `.claude/` define o contexto permanente utilizado pelo Claude Code dentro da plataforma TAPOS, garantindo que o desenvolvimento siga padrões consistentes de arquitetura, engenharia e evolução incremental — transformando o Claude em um assistente orientado por contexto, não apenas um gerador de código.

Conteúdo atual

```
.claude/
├── workspace.md           # arquitetura e estrutura da plataforma
├── constitution.md       # regras e princípios de engenharia
├── current-phase.md      # foco e etapa atual do projeto
├── settings.local.json   # permissões locais de ferramentas
├── scheduled_tasks.lock  # lock de execução em runtime
├── skills/              # reservado – vazio
├── agents/              # reservado – vazio
├── rules/               # reservado – vazio
└── memory/             # reservado – vazio
```

Como o Claude usa esta pasta

Ao iniciar o Claude Code a partir da raiz do workspace (`/workspace/tecle`), ele passa a utilizar automaticamente:

- **arquitetura** ([workspace.md](#) / `.claude/workspace.md`)
- **regras** ([constitution.md](#) / `.claude/constitution.md`)
- **foco atual** (`.claude/current-phase.md`)

Estratégia de expansão planejada

Quatro diretórios foram criados antecipadamente como scaffolding para fases futuras, e devem permanecer vazios até que sejam explicitamente necessários:

- `agents/` → agentes especializados
- `skills/` → capacidades reutilizáveis
- `memory/` → contexto persistente
- `rules/` → regras adicionais

A regra explícita é que **estes diretórios só devem ser criados/populados após estabilidade do backend SaaS** — o que hoje, com a baseline v1.0.0 operacional (autenticação, produtos, assinaturas, gateway, execução assíncrona), é uma condição já atendida, tornando esta expansão um próximo passo natural e não mais prematuro.

`current-phase.md` **está desatualizado**

Vale registrar com clareza: `.claude/current-phase.md` descreve o estágio "SaaS Backend Initialization", com a tarefa atual sendo "criar estrutura inicial do backend com FastAPI, app base e endpoint `/health`", e lista explicitamente como **não permitido** neste momento: "criar auth completo", "integrar outros serviços". Esta é uma fotografia da fase mais inicial do projeto.

O `changelog.md` da raiz, porém, já documenta a **v1.0.0**, com JWT Authentication, Products, Subscriptions, Authorization, Product Gateway e Speech-AI integrado (tarefas 014A/014B concluídas) — várias fases à frente do que `current-phase.md` descreve. Ou seja: **`current-phase.md` nunca foi atualizado após a primeira tarefa** e não deve ser tratado como fonte de verdade sobre o estágio atual da plataforma. Até que seja atualizado, o `changelog.md` e o histórico em `tasks/saas/` são as fontes corretas sobre o que já foi construído.

Regras importantes de uso

- Sempre iniciar o Claude na raiz `/workspace/tecle`.
- Nunca trabalhar em subpastas isoladas.
- Nunca gerar múltiplos módulos ao mesmo tempo.
- Nunca pular validação.
- Sempre commitar após cada etapa (ver [development-flow.md](#) para o quanto essa regra foi seguida na prática até agora).

Ver também

- [workspace.md](#) — estrutura completa do workspace que este contexto governa
- [constitution.md](#) — as regras que o Claude Code aplica
- [development-flow.md](#) — o ciclo de desenvolvimento na prática

Fluxo de Desenvolvimento

O ciclo prescrito

Todo o desenvolvimento da TAPOS segue, por princípio, um ciclo disciplinado e incremental:

```
Definir tarefa
↓
Implementar (1 módulo)
↓
Testar
↓
Commit
↓
Próxima tarefa
```

O template real de tarefa

Cada tarefa vive em `tasks/saas/NNN-nome/` e segue um template consistente, observado nas 12 pastas de `005` a `015`:

- **task.md** — Contexto / Objetivo / Requisitos / Restrições / Tratamento de erro / Notas — incluindo o que **não** deve ser feito ainda naquela tarefa.
- **acceptance.md** — critérios funcionais e técnicos, com comandos `curl` reais e executáveis que provam que o endpoint funciona.
- **notes.md** — decisões tomadas e o que foi explicitamente adiado para uma tarefa futura.

Exemplo concreto da disciplina de adiamento: as notas da tarefa `007-login` dizem explicitamente "Future: adicionar JWT (Task 008), criar `/users/me`, implementar controle de acesso" — e de fato a tarefa `008` é `008-jwt` e a `009` é `009-protected-route`. As notas da tarefa `013-authorization` adiam, "middleware/dependency reutilizável", "integração com speech-ai" e "integração com edital-ai" — e `014a` / `014b` são exatamente essas integrações. O encadeamento das tarefas no repositório é evidência direta de que o ciclo Definir → Implementar → Testar → Commit → Próximo foi seguido no *planejamento e escopo* de cada etapa.

Histórico de tarefas (005 → 015)

Tarefa	Entregue
005-password-hash	Hash de senha com bcrypt
006-register-hash	Endpoint de registro de usuário
007-login	Login com validação de credenciais
008-jwt	Geração de JWT

Tarefa	Entregue
009-protected-route	Rota protegida (/users/me) com JWT
010-user-profile	Perfil de usuário enriquecido
011-products	Modelo de produtos (slug por vertical)
012-subscriptions	Assinaturas ativas/inativas por produto
013-authorization	Autorização central (produto + assinatura)
014-product-gateway	Gateway único de acesso a produtos
014a-speech-ai-facade	Contrato facade/runner/schemas/cli do Speech-AI
014b-tapos-speech-ai-integration	Speech-AI integrado de ponta a ponta na plataforma
015-async-speech-ai-execution	Execução assíncrona real via fila e worker

A realidade dos commits — uma divergência honesta

A constituição e o `workspace.md` prescrevem "commit constante" e "comitar após cada etapa". Na prática, até o momento desta documentação, **todo o histórico do repositório é um único commit**:

```
a48b7b5 TAP0S v1.0.0 - operational SaaS baseline
```

Esse commit único reúne 359 arquivos alterados e mais de 113 mil linhas inseridas — todo o `.claude/`, o backend completo, os três produtos, documentação e até artefatos binários (PDFs, imagens, um `.docx`, o áudio do pitch para investidores). Isso significa que o trabalho de todas as 12 tarefas (005–015) foi acumulado localmente e consolidado em um único commit de baseline, em vez de commits incrementais por módulo, como a constituição prescreve.

Isso não invalida o princípio — é um registro honesto de que a disciplina de commit precisa ser aplicada de forma mais rigorosa a partir daqui, tarefa por tarefa, à medida que a v1.1 e versões seguintes forem desenvolvidas.

Testes reais

Os testes que efetivamente existem e rodam vivem em `platform/saas-backend/tests/` (`test_deps.py`, `test_security.py`), não na pasta `tests/` da raiz, que está vazia (reservada). Os critérios de aceite de cada tarefa (`acceptance.md`) também funcionam como uma camada de validação funcional via `curl`, complementar aos testes automatizados.

Artefatos de descarte

A pasta `trash/` contém material de diagnóstico e depuração pontual (capturas de rede, transcrições de shell de debug, exports forenses, scripts de teste manuais como `test_auth.sh` e `test_task015.sh`) — não faz parte do fluxo formal de tarefas, mas foi preservada em vez de descartada, por precaução.

Ver também

- [claude-code.md](#) — como o Claude Code opera dentro deste ciclo
- [constitution.md](#) — os princípios que este fluxo implementa
- [../01-tapos/roadmap.md](#) — o que vem depois da tarefa 015